

```
In [29]: import subprocess
import sys

# Crear un entorno virtual llamado 'venv'
subprocess.check_call([sys.executable, "-m", "venv", "venv"])
print("Entorno virtual creado correctamente")
```

Entorno virtual creado correctamente

<h1 id="%F0%9F%93%8A-Evaluaci%C3%B3n-Sem%C3%A1ntica-de-Pictogramas"> 

Evaluación Semántica de Pictogramas</h1>

<h1 id="1.-Instalaci%C3%B3n-y-dependencias">1. Instalación y dependencias</h1>

```
In [30]: # Instalación de dependencias
%pip install sacrebleu scipy scikit-learn matplotlib seaborn pandas numpy krippe
```

Note: you may need to restart the kernel to use updated packages.

```
In [31]: import subprocess, sys
packages = ["sacrebleu", "scipy", "scikit-learn", "matplotlib", "seaborn", "pand
for pkg in packages:
    subprocess.check_call([sys.executable, "-m", "pip", "install", pkg, "-q"])
print("Dependencias instaladas correctamente")
```

Dependencias instaladas correctamente

```
In [32]: import json
import os
import re
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from collections import Counter
from sacrebleu.metrics import BLEU, CHRF
from scipy.stats import pearsonr, spearmanr
from sklearn.metrics import f1_score
from sklearn.metrics.pairwise import cosine_similarity
from sentence_transformers import SentenceTransformer
import warnings
warnings.filterwarnings('ignore')
# Estilo visual
plt.rcParams['figure.figsize'] = (10, 5)
plt.rcParams['font.size'] = 12
sns.set_style("whitegrid")
sns.set_palette("husl")
print("Imports correctos")
```

Imports correctos

<h1 id="2.-Carga-de-datos">2. Carga de datos</h1>

```
In [33]: # -----
# AJUSTA ESTA RUTA a donde tengas tus archivos de datos
# -----
DATA_DIR = "data" # ← carpeta data/ está un nivel arriba de notebooks/

def load_json(filename):
    path = os.path.join(DATA_DIR, filename)
```

```

    with open(path, "r", encoding="utf-8") as f:
        return json.load(f)

# Dataset principal
train_data = load_json("train.json")
test_data = load_json("test (2).json")
val_data = load_json("validation.json")

# Predicciones de Los 4 modelos
modelo1 = load_json("modelo1.json")
modelo2 = load_json("modelo2.json")
modelo3 = load_json("modelo3.json")
modelo4 = load_json("modelo4.json")

# IDs de pictogramas válidos
with open(os.path.join(DATA_DIR, "ids_only_id.txt"), "r") as f:
    valid_pict_ids = set(line.strip() for line in f.readlines())

# Evaluaciones humanas
EVALUATOR_FILES = [
    "respuestas_a.paredes.um@gmail.com",
    "respuestas_angela.mariategui@gmail.com",
    "respuestas_CUSIPAUCARALACANTARAGMAIL.COM",
    "respuestas_humansanchezmayra@gmail.com",
    "respuestas_ilivaldez@hotmail.com",
    "respuestas_k.leyvarivera@gmail.com",
    "respuestas_norka.bringas@gmail.com",
    "respuestas_psico.espinoza18@gmail.com",
]
evaluations = []

for fname in EVALUATOR_FILES:
    path = os.path.join(DATA_DIR, fname)
    with open(path, "r", encoding="utf-8") as f:
        evaluations.append(json.load(f))

print(f"train: {len(train_data):,} ejemplos")
print(f"test: {len(test_data):,} ejemplos")
print(f"val: {len(val_data):,} ejemplos")
print(f"IDs de pictogramas válidos: {len(valid_pict_ids):,}")
print(f"Modelos cargados: {len(modelo1)} predicciones c/u")
print(f"Evaluadores humanos: {len(evaluations)}")

```

```

train: 15,208 ejemplos
test: 973 ejemplos
val: 973 ejemplos
IDs de pictogramas válidos: 13,515
Modelos cargados: 50 predicciones c/u
Evaluadores humanos: 8

```

```

In [34]: # Crear diccionarios indexados por id para acceso rápido
test_by_id = {x["id"]: x for x in test_data}
model1_by_id = {x["id"]: x for x in modelo1}
model2_by_id = {x["id"]: x for x in modelo2}
model3_by_id = {x["id"]: x for x in modelo3}
model4_by_id = {x["id"]: x for x in modelo4}
# IDs evaluados (50 ejemplos)
EVAL_IDS = sorted(model1_by_id.keys())
print(f"IDs evaluados: {EVAL_IDS[:5]}...{EVAL_IDS[-5:]} (total: {len(EVAL_IDS)})")

```

IDs evaluados: [0, 1, 2, 3, 4]...[968, 969, 970, 971, 972] (total: 50)

<h1 id="3.-An%C3%A1lisis-exploratorio-del-dataset">3. Análisis exploratorio del dataset</h1>

```
In [35]: # — Estadísticas básicas del corpus —————
def token_stats(data, field="oracion"):
    lengths = [len(x[field].split()) for x in data if field in x]
    return {
        "n": len(lengths),
        "min": min(lengths),
        "max": max(lengths),
        "mean": round(np.mean(lengths), 2),
        "std": round(np.std(lengths), 2),
        "median": int(np.median(lengths))
    }

def pict_stats(data, field="traduccion"):
    lengths = [len(x[field].split()) for x in data if field in x]
    return {
        "n": len(lengths),
        "min": min(lengths),
        "max": max(lengths),
        "mean": round(np.mean(lengths), 2),
        "std": round(np.std(lengths), 2),
        "median": int(np.median(lengths))
    }

splits = {"train": train_data, "test": test_data, "validation": val_data}
stats_rows = []
for name, data in splits.items():
    ts = token_stats(data, "oracion")
    ps = pict_stats(data, "traduccion")
    stats_rows.append({
        "Split": name,
        "N ejemplos": ts["n"],
        "Tokens/frase (media)": ts["mean"],
        "Tokens/frase (max)": ts["max"],
        "Pictogramas/ref (media)": ps["mean"],
        "Pictogramas/ref (max)": ps["max"],
    })

df_stats = pd.DataFrame(stats_rows)
print("=== Estadísticas del corpus ===")
print(df_stats.to_string(index=False))
```

```
=== Estadísticas del corpus ===
Split N ejemplos Tokens/frase (media) Tokens/frase (max) Pictogramas/ref
(media) Pictogramas/ref (max)
train 15208 4.76 15
3.91 14
test 973 5.09 22
3.15 11
validation 973 4.99 21
3.08 11
```

```
In [36]: # — Distribución de Longitudes en train —————
fig, axes = plt.subplots(1, 2, figsize=(14, 5))

oracion_lens = [len(x["oracion"].split()) for x in train_data]
```

```

pict_lens = [len(x["traduccion"].split()) for x in train_data]

# Gráfico 1: oraciones
axes[0].hist(oracion_lens, bins=40, color="#4C72B0", edgecolor="white", alpha=0.8)
axes[0].set_title("Distribución: longitud de frases (train)")
axes[0].set_xlabel("Número de palabras")
axes[0].set_ylabel("Frecuencia")
axes[0].axvline(
    np.mean(oracion_lens),
    color="red",
    linestyle="--",
    label=f"Media: {np.mean(oracion_lens):.2f}"
)
axes[0].legend()

# Gráfico 2: pictogramas
axes[1].hist(pict_lens, bins=40, color="#DD8452", edgecolor="white", alpha=0.85)
axes[1].set_title("Distribución: longitud de secuencias pictográficas (train)")
axes[1].set_xlabel("Número de pictogramas")
axes[1].set_ylabel("Frecuencia")
axes[1].axvline(
    np.mean(pict_lens),
    color="red",
    linestyle="--",
    label=f"Media: {np.mean(pict_lens):.2f}"
)
axes[1].legend()

plt.tight_layout()
plt.savefig("dist_longitudes.png", dpi=150, bbox_inches="tight")
plt.show()

print("Gráfico guardado: dist_longitudes.png")

```

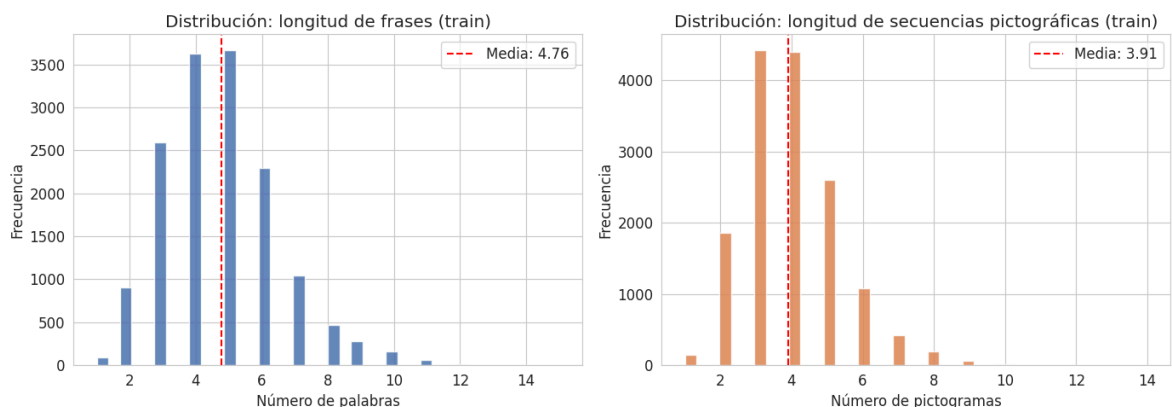


Gráfico guardado: dist_longitudes.png

```

In [37]: # — Pictogramas más frecuentes —————
all_picts = []
for x in train_data:
    all_picts.extend(x["traduccion"].split())

pict_counter = Counter(all_picts)
top20 = pict_counter.most_common(20)

df_top = pd.DataFrame(top20, columns=["Pictograma", "Frecuencia"])

print("=== Top 20 pictogramas más frecuentes (train) ===")
print(df_top.to_string(index=False))

```

```
# Plot
fig, ax = plt.subplots(figsize=(12, 6))

ax.bar(
    [p for p, _ in top20],
    [c for _, c in top20],
    color="#55A868",
    edgecolor="white"
)

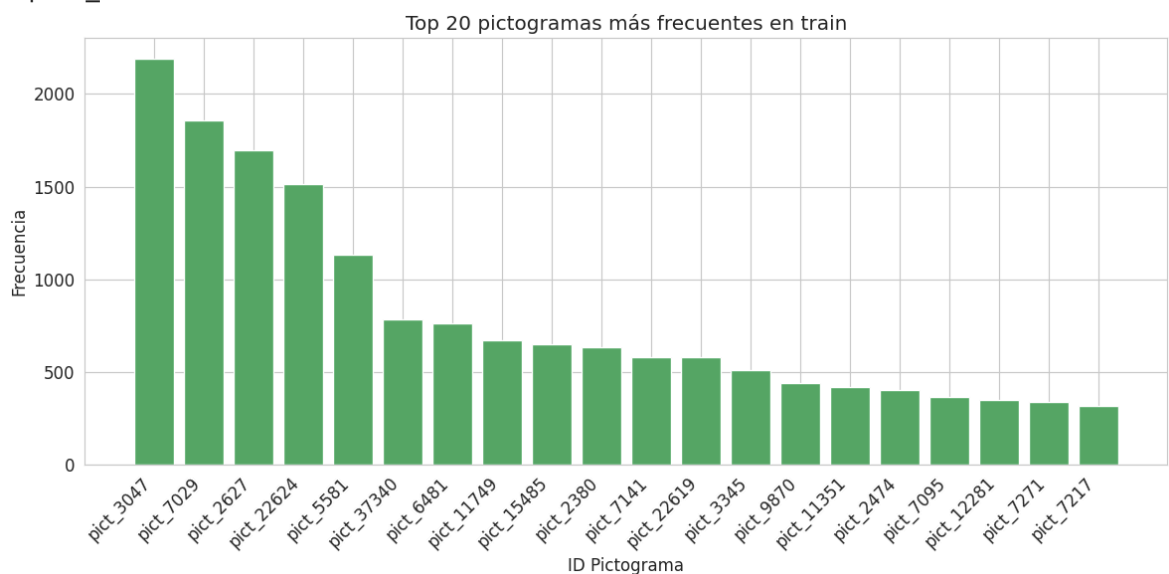
ax.set_title("Top 20 pictogramas más frecuentes en train")
ax.set_xlabel("ID Pictograma")
ax.set_ylabel("Frecuencia")

plt.xticks(rotation=45, ha="right")

plt.tight_layout()
plt.savefig("top_pictogramas.png", dpi=150, bbox_inches="tight")
plt.show()
```

=== Top 20 pictogramas más frecuentes (train) ===

Pictograma	Frecuencia
pict_3047	2192
pict_7029	1858
pict_2627	1698
pict_22624	1515
pict_5581	1132
pict_37340	784
pict_6481	760
pict_11749	668
pict_15485	646
pict_2380	630
pict_7141	581
pict_22619	576
pict_3345	511
pict_9870	441
pict_11351	420
pict_2474	399
pict_7095	362
pict_12281	348
pict_7271	337
pict_7217	317



```
In [38]: # — Vocabulario y cobertura —————
train_picts = set()
for x in train_data:
    train_picts.update(x["traduccion"].split())

test_picts = set()
for x in test_data:
    if "traduccion" in x:
        test_picts.update(x["traduccion"].split())

# Tokens fuera de vocabulario (no son pict_xxxx)
def count_oov(data, field="traduccion"):
    oov = set()
    for x in data:
        if field in x:
            for tok in x[field].split():
                if not tok.startswith("pict_"):
                    oov.add(tok)
    return oov

train_oov = count_oov(train_data)
test_oov = count_oov(test_data)

print(f"Vocabulario (pictogramas únicos):")
print(f" train: {len(train_picts):,} | test: {len(test_picts):,}")
print(f"\nTokens NO-pictograma (palabras textuales) en referencias:")
print(f" train OOV: {len(train_oov):,} | test OOV: {len(test_oov):,}")
print(f" Ejemplos de OOV en test: {list(test_oov)[:10]}")
```

Vocabulario (pictogramas únicos):
train: 5,900 | test: 1,276

Tokens NO-pictograma (palabras textuales) en referencias:
train OOV: 3,962 | test OOV: 0
Ejemplos de OOV en test: []

<h1 id="4.-Limpieza-de-datos">4. Limpieza de datos</h1>

```
In [39]: # — Funciones de limpieza —————
def clean_sequence(seq_str):
    """Limpia una secuencia de pictogramas:
    - Normaliza espacios
    - Elimina tokens vacíos
    - Devuelve lista de tokens
    """
    if not seq_str or not isinstance(seq_str, str):
        return []

    tokens = seq_str.strip().split()

    # Quitar tokens vacíos o solo espacios
    tokens = [t.strip() for t in tokens if t.strip()]

    return tokens

def is_valid_pict_token(tok):
    """True si el token es un pict_ID válido."""
    return tok.startswith("pict_") and tok in valid_pict_ids
```

```

def sequence_coverage(tokens):
    """Porcentaje de tokens que son pictogramas válidos."""
    if not tokens:
        return 0.0

    valid = sum(1 for t in tokens if is_valid_pict_token(t))
    return valid / len(tokens)

# — Auditoría de predicciones —————
models = {
    "Modelo 1": model1_by_id,
    "Modelo 2": model2_by_id,
    "Modelo 3": model3_by_id,
    "Modelo 4": model4_by_id
}

audit_rows = []

for m_name, m_dict in models.items():
    pict_valid, pict_total, oov_count = 0, 0, 0
    n_empty = 0

    for sid in EVAL_IDS:
        tokens = clean_sequence(m_dict[sid]["output"])

        if len(tokens) == 0:
            n_empty += 1

        for tok in tokens:
            pict_total += 1
            if is_valid_pict_token(tok):
                pict_valid += 1
            else:
                oov_count += 1

    audit_rows.append({
        "Modelo": m_name,
        "Predicciones vacías": n_empty,
        "Tokens totales": pict_total,
        "pict_ID válidos": pict_valid,
        "Tokens OOV / texto": oov_count,
        "% Cobertura": round(100 * pict_valid / pict_total if pict_total else 0,
    })

df_audit = pd.DataFrame(audit_rows)

print("=== Auditoría de predicciones ===")
print(df_audit.to_string(index=False))

```

=== Auditoría de predicciones ===

Modelo	Predicciones vacías	Tokens totales	pict_ID válidos	Tokens OOV / text
Modelo 1	0	201	189	1
2	94.0			
Modelo 2	0	204	184	2
0	90.2			
Modelo 3	0	204	204	
0	100.0			
Modelo 4	0	167	167	
0	100.0			

```
In [40]: # — Auditoría de referencias (ground truth del test subset) —
ref_empty = 0
ref_oov_tokens = []
for sid in EVAL_IDS:
    ref_tokens = clean_sequence(test_by_id[sid]["traduccion"])
    if len(ref_tokens) == 0:
        ref_empty += 1
    for tok in ref_tokens:
        if not tok.startswith("pict_"):
            ref_oov_tokens.append(tok)
print(f"Referencias vacías: {ref_empty}")
print(f"Tokens OOV en referencias: {len(ref_oov_tokens)}")
print(f"Tokens OOV únicos: {len(set(ref_oov_tokens))}")
print(f"Ejemplos: {list(set(ref_oov_tokens))[:15]}")
```

Referencias vacías: 0

Tokens OOV en referencias: 0

Tokens OOV únicos: 0

Ejemplos: []

<h1 id="5.-M%C3%A9tricas-baseline:-BLEU-y-chrF++">5. Métricas baseline: BLEU y chrF++</h1>

```
In [41]: # — Preparar hipótesis y referencias —
def prepare_sentences(model_dict, ref_dict, ids):
    """Retorna (hypotheses_str, references_str) como listas de strings."""
    hyps, refs = [], []
    for sid in ids:
        hyp = " ".join(clean_sequence(model_dict[sid]["output"]))
        ref = " ".join(clean_sequence(ref_dict[sid]["traduccion"]))
        hyps.append(hyp)
        refs.append(ref)
    return hyps, refs

# Instanciar métricas
bleu_metric = BLEU(effective_order=True)
chrF_metric = CHRF(word_order=2) # chrF++ (word_order=2)

metric_rows = []
for m_name, m_dict in models.items():
    hyps, refs = prepare_sentences(m_dict, test_by_id, EVAL_IDS)

    #
    # | NOTA: corpus_score vs sentence_score en sacrebleu |
    # |
    # | corpus_score(hyps, [refs]) |
    # | → Calcula BLEU sobre el corpus completo en un solo pase. |
```



```

# → Usa n-gram counts acumulados sobre todos los ejemplos.
# → Aplica brevity penalty global.
# → USO: reporte general y tabla comparativa de modelos.
# → Es el estándar en papers (más estable estadísticamente).
#
# sentence_score(hyp, [ref]) - ver Sección 8
# → Calcula BLEU para una sola oración.
# → Usa effective_order=True para evitar log(0) en frases
# cortas sin coincidencias en n-gramas de orden alto.
# → USO: correlación per-sample con evaluación humana.
# → Alta varianza en frases cortas; NO usar para ranking
# global de modelos.
#
# BLEU espera: hypotheses, [references]
bleu_score = bleu_metric.corpus_score(hyps, [refs])
chrF_score = chrF_metric.corpus_score(hyps, [refs])

metric_rows.append({
    "Modelo": m_name,
    "BLEU": round(bleu_score.score, 2),
    "chrF++": round(chrF_score.score, 2),
})
print(f"{m_name}: BLEU={bleu_score.score:.2f} chrF++={chrF_score.score:.2f}")

df_metrics = pd.DataFrame(metric_rows)
print("\n=== Tabla resumen métricas baseline ===")
print(df_metrics.to_string(index=False))

```

```

Modelo 1: BLEU=41.24 chrF++=55.73
Modelo 2: BLEU=29.68 chrF++=43.40
Modelo 3: BLEU=32.30 chrF++=48.75
Modelo 4: BLEU=30.14 chrF++=41.88

```

```

=== Tabla resumen métricas baseline ===
Modelo BLEU chrF++
Modelo 1 41.24 55.73
Modelo 2 29.68 43.40
Modelo 3 32.30 48.75
Modelo 4 30.14 41.88

```

```

In [42]: # — Visualización de métricas baseline —————
fig, axes = plt.subplots(1, 2, figsize=(12, 5))

x = np.arange(len(df_metrics))
w = 0.5
colors = sns.color_palette("husl", 4)

for i, (col, title) in enumerate([("BLEU", "BLEU Score"), ("chrF++", "chrF++ Score")]):
    ax2 = axes[i]

    bars = ax2.bar(
        df_metrics["Modelo"],
        df_metrics[col],
        color=colors,
        edgecolor="white"
    )

    ax2.set_title(title)
    ax2.set_ylabel("Score")
    ax2.set_ylim(0, max(df_metrics[col]) * 1.3)

```

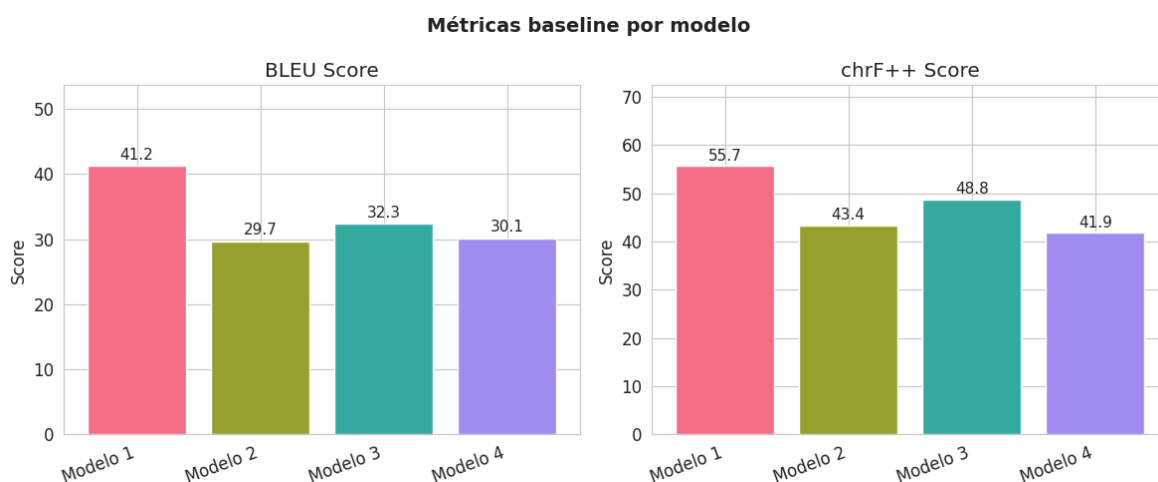
```

# Etiquetas encima de las barras
for bar in bars:
    ax2.text(
        bar.get_x() + bar.get_width() / 2,
        bar.get_height() + 0.5,
        f"{bar.get_height():.1f}",
        ha="center",
        va="bottom",
        fontsize=11
    )

plt.setp(ax2.get_xticklabels(), rotation=20, ha="right")

plt.suptitle("Métricas baseline por modelo", fontsize=14, fontweight="bold")
plt.tight_layout()
plt.savefig("metricas_baseline.png", dpi=150, bbox_inches="tight")
plt.show()

```



<h1 id="6.-M%C3%A9tricas-sem%C3%A1nticas:-Concept-F1,-Coverage-Score-y-Semantic-Similarity">6. Métricas semánticas: Concept F1, Coverage Score y Semantic Similarity</h1>

```

In [43]: # — Concept F1 —————
# Compara conjuntos de tokens (bag-of-pictograms) sin importar el orden

def concept_precision_recall_f1(hyp_tokens, ref_tokens):
    """Calcula P, R, F1 basado en conjuntos de pictogramas."""

    hyp_set = set(hyp_tokens)
    ref_set = set(ref_tokens)

    # Casos borde
    if len(hyp_set) == 0 and len(ref_set) == 0:
        return 1.0, 1.0, 1.0
    if len(hyp_set) == 0 or len(ref_set) == 0:
        return 0.0, 0.0, 0.0

    tp = len(hyp_set & ref_set)

    precision = tp / len(hyp_set)
    recall = tp / len(ref_set)

    # Evitar división por cero

```

```

    if (precision + recall) == 0:
        f1 = 0.0
    else:
        f1 = 2 * precision * recall / (precision + recall)

    return precision, recall, f1

# — Concept F1 para cada modelo —————
semantic_rows = []

for m_name, m_dict in models.items():
    p_list, r_list, f1_list = [], [], []

    for sid in EVAL_IDS:
        hyp_tokens = clean_sequence(m_dict[sid]["output"])
        ref_tokens = clean_sequence(test_by_id[sid]["traduccion"])

        p, r, f1 = concept_precision_recall_f1(hyp_tokens, ref_tokens)

        p_list.append(p)
        r_list.append(r)
        f1_list.append(f1)

    semantic_rows.append({
        "Modelo": m_name,
        "Concept P": round(np.mean(p_list) * 100, 2),
        "Concept R": round(np.mean(r_list) * 100, 2),
        "Concept F1": round(np.mean(f1_list) * 100, 2),
    })

df_semantic = pd.DataFrame(semantic_rows)

print("=== Concept F1 por modelo ===")
print(df_semantic.to_string(index=False))

```

```

=== Concept F1 por modelo ===
  Modelo  Concept P  Concept R  Concept F1
Modelo 1      42.34      48.77      44.49
Modelo 2      24.86      27.20      25.49
Modelo 3      35.00      40.07      36.62
Modelo 4      30.03      30.60      28.88

```

```

In [44]: # — Coverage Score —————
# ¿Qué porcentaje de pictogramas de la referencia aparece en la predicción?

def coverage_score(hyp_tokens, ref_tokens):
    """Fracción de pictos de referencia cubiertos por la predicción."""

    if not ref_tokens:
        return 1.0

    hyp_set = set(hyp_tokens)
    covered = sum(1 for t in ref_tokens if t in hyp_set)

    return covered / len(ref_tokens)

# — Calcular coverage por modelo —————
coverage_rows = []

```

```

for m_name, m_dict in models.items():
    cov_list = []

    for sid in EVAL_IDS:
        hyp_tokens = clean_sequence(m_dict[sid]["output"])
        ref_tokens = clean_sequence(test_by_id[sid]["traduccion"])

        cov_list.append(coverage_score(hyp_tokens, ref_tokens))

    coverage_rows.append({
        "Modelo": m_name,
        "Coverage": round(np.mean(cov_list) * 100, 2)
    })

df_cov = pd.DataFrame(coverage_rows)

print("=== Coverage Score por modelo ===")
print(df_cov.to_string(index=False))

```

```
=== Coverage Score por modelo ===
```

Modelo	Coverage
Modelo 1	48.77
Modelo 2	27.20
Modelo 3	40.07
Modelo 4	30.60

```

In [45]: # — Similaridad semántica con embeddings —————

# — Construir diccionario de idPictograma - concepto —————
print("Construyendo lookup de pictogramas...")
pict_lookup = {}

arasaac_data = load_json("pictogramasArasaac.json")

for p in arasaac_data:
    pid = str(p["_id"])

    keyword_found = ""
    if "keywords" in p and isinstance(p["keywords"], list) and len(p["keywords"]) > 0:
        keyword_found = p["keywords"][0].get("keyword", "").lower()

    if keyword_found:
        pict_lookup[pid] = keyword_found

print(f"Pictogramas en lookup: {len(pict_lookup):,}")

# — Función de conversión tokens → texto —————
def tokens_to_text(token_list):
    """Convierte lista de pict_XXXX a string de keywords reales."""
    words = []
    for tok in token_list:
        if tok.startswith("pict_"):
            pid = tok.replace("pict_", "")
            word = pict_lookup.get(pid)
            if word:
                words.append(word)
    return " ".join(words) if words else ""

# — Cargar modelo de embeddings —————

```

```

print("Cargando modelo de embeddings (descarga ~117MB la primera vez)...")
model_emb = SentenceTransformer("paraphrase-multilingual-MiniLM-L12-v2")
print("Modelo cargado exitosamente.")

# — Consequir similitud entres 2 textos —————
def get_semantic_similarity(ref_text, hyp_text):
    """Calcula la similitud de coseno entre dos textos usando el modelo cargado.
    if not hyp_text or not ref_text:
        return 0.0

    v1 = model_emb.encode([ref_text])
    v2 = model_emb.encode([hyp_text])

    sim = cosine_similarity(v1, v2)[0][0]
    return float(sim * 100)

# — Calcular similaridad por modelo —————
sem_sim_rows = []

for m_name, m_dict in models.items():
    sim_list = []
    for sid in EVAL_IDS:
        oracion = test_by_id[sid]["oracion"]
        hyp_tokens = clean_sequence(m_dict[sid]["output"])
        hyp_text = tokens_to_text(hyp_tokens)

        sim = get_semantic_similarity(oracion, hyp_text)
        sim_list.append(sim)

    mean_sim = round(np.mean(sim_list), 2)
    sem_sim_rows.append({
        "Modelo": m_name,
        "Semantic Similarity": mean_sim,
    })

df_sem_sim = pd.DataFrame(sem_sim_rows)
print("\n=== Similaridad semántica promedio por modelo ===")
print(df_sem_sim.to_string(index=False))

# — Tabla consolidada —————
df_all_metrics = (
    df_metrics
    .merge(df_semantic, on="Modelo")
    .merge(df_cov, on="Modelo")
    .merge(df_sem_sim, on="Modelo")
)

print("\n=== TABLA CONSOLIDADA DE MÉTRICAS ===")
print(df_all_metrics.to_string(index=False))

```

Construyendo lookup de pictogramas...

Pictogramas en lookup: 13,515

Cargando modelo de embeddings (descarga ~117MB la primera vez)...

Loading weights: 0% | 0/199 [00:00<?, ?it/s]

Modelo cargado exitosamente.

=== Similaridad semántica promedio por modelo ===

Modelo	Semantic Similarity
Modelo 1	77.86
Modelo 2	61.97
Modelo 3	81.24
Modelo 4	79.28

=== TABLA CONSOLIDADA DE MÉTRICAS ===

Modelo	BLEU	chrF++	Concept P	Concept R	Concept F1	Coverage	Semantic Similarity
Modelo 1	41.24	55.73	42.34	48.77	44.49	48.77	77.86
Modelo 2	29.68	43.40	24.86	27.20	25.49	27.20	61.97
Modelo 3	32.30	48.75	35.00	40.07	36.62	40.07	81.24
Modelo 4	30.14	41.88	30.03	30.60	28.88	30.60	79.28

```
In [46]: # — Visualización: Concept F1, Coverage y Semantic Similarity —————
fig, axes = plt.subplots(1, 3, figsize=(16, 5))
metric_cols = ["Concept F1", "Coverage", "Semantic Similarity"]
colors = sns.color_palette("husl", 4)

for i, col in enumerate(metric_cols):
    ax = axes[i]

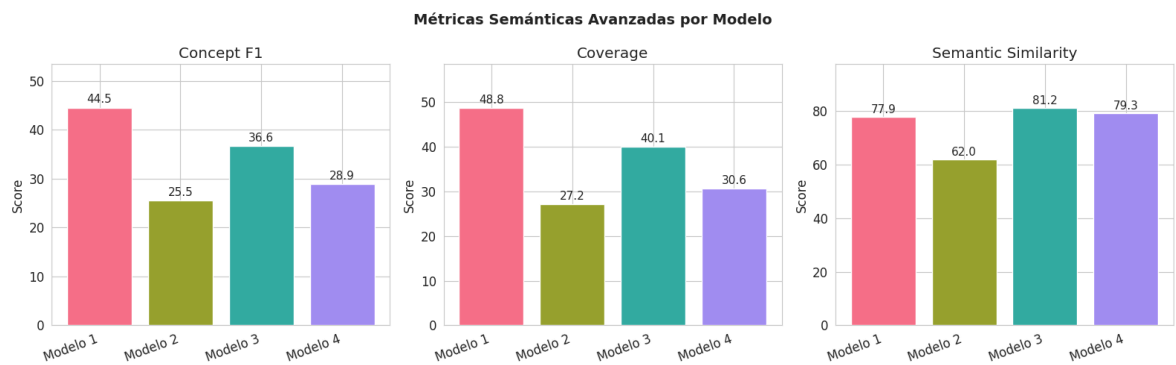
    bars = ax.bar(
        df_all_metrics["Modelo"],
        df_all_metrics[col],
        color=colors,
        edgecolor="white",
    )

    ax.set_title(col)
    ax.set_ylabel("Score")
    ax.set_ylim(0, max(df_all_metrics[col]) * 1.2)

    # Etiquetas encima de las barras
    for bar in bars:
        ax.text(
            bar.get_x() + bar.get_width() / 2,
            bar.get_height() + 0.5,
            f"{bar.get_height():.1f}",
            ha="center", va="bottom", fontsize=11
        )

    plt.setp(ax.get_xticklabels(), rotation=20, ha="right")

plt.suptitle("Métricas Semánticas Avanzadas por Modelo", fontsize=14, fontweight="bold")
plt.tight_layout()
plt.savefig("metricas_semanticas_avanzadas.png", dpi=150, bbox_inches="tight")
plt.show()
```



```
In [47]: # — Grouped Bar Chart – reemplaza el heatmap —————
# Grouped Bar Chart → cada métrica tiene su propio grupo de
# barras; La comparación se hace ENTRE MODELOS dentro de cada métrica,
# que sí es válida (todos usan la misma escala 0-100 para esa métrica).
# —————

import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

# — Configuración —————
METRICS_PLOT = ["BLEU", "chrF++", "Concept F1", "Coverage", "Semantic Similarity"]
MODELOS      = df_all_metrics["Modelo"].tolist()
N_METRICS    = len(METRICS_PLOT)
N_MODELS     = len(MODELOS)
colors       = sns.color_palette("husl", N_MODELS)
x            = np.arange(N_METRICS)
width        = 0.18

# — Figura —————
fig, ax = plt.subplots(figsize=(14, 6))

for i, (modelo, color) in enumerate(zip(MODELOS, colors)):
    row = df_all_metrics[df_all_metrics["Modelo"] == modelo].iloc[0]
    values = [row[m] for m in METRICS_PLOT]
    offset = (i - N_MODELS / 2 + 0.5) * width
    bars = ax.bar(
        x + offset, values, width,
        label=modelo, color=color, edgecolor="white", linewidth=0.8
    )
    for bar, val in zip(bars, values):
        ax.text(
            bar.get_x() + bar.get_width() / 2,
            bar.get_height() + 0.6,
            f"{val:.1f}",
            ha="center", va="bottom", fontsize=9, fontweight="bold"
        )

# — Anotación discrepancia chrF++ M2 vs M4 —————
chrF_idx = METRICS_PLOT.index("chrF++")
m2_chrf = df_all_metrics[df_all_metrics["Modelo"] == "Modelo 2"]["chrF++"].value
ax.annotate(
    "⚠ M2 > M4\ñen chrF++\ñ(ver análisis §6)",
    xy=(chrF_idx + (1 - N_MODELS / 2 + 0.5) * width, m2_chrf),
    xytext=(chrF_idx + 0.58, m2_chrf + 7),
    fontsize=8, color="#c0392b",
    arrowprops=dict(arrowstyle="->", color="#c0392b", lw=1.2),
    bbox=dict(boxstyle="round,pad=0.3", fc="white", ec="#c0392b", alpha=0.85)
```

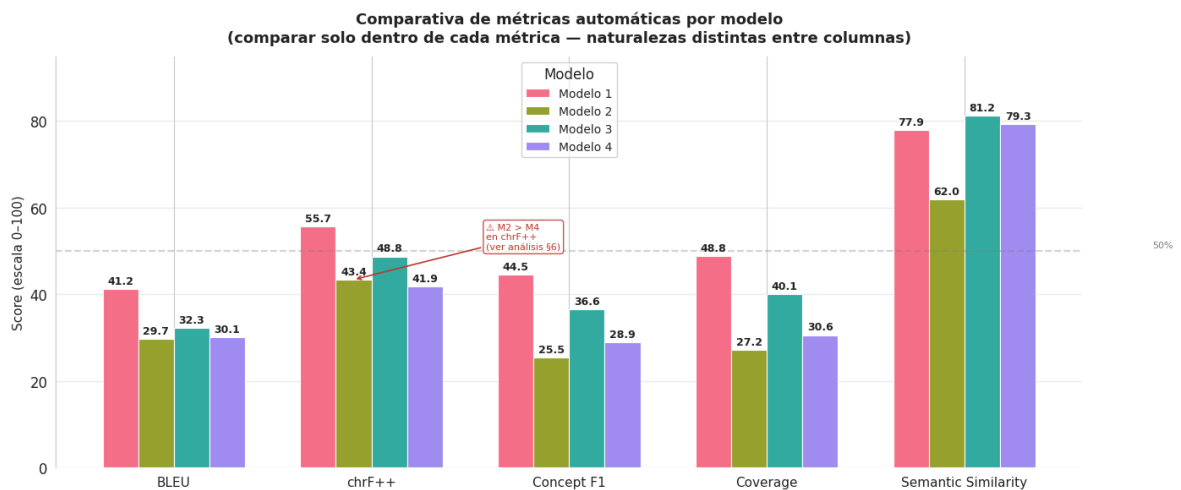
```

)

# — Estética —————
ax.set_xticks(x)
ax.set_xticklabels(METRICS_PLOT, fontsize=11)
ax.set_ylabel("Score (escala 0-100)", fontsize=11)
ax.set_ylim(0, 95)
ax.set_title(
    "Comparativa de métricas automáticas por modelo\n"
    "(comparar solo dentro de cada métrica – naturalezas distintas entre columna"
    fontsize=13, fontweight="bold", pad=12
)
ax.legend(title="Modelo", loc="upper center", fontsize=10)
ax.axhline(50, color="gray", linestyle="--", alpha=0.35)
ax.text(N_METRICS - 0.05, 50.8, "50%", color="gray", fontsize=8)
ax.grid(axis="y", alpha=0.3)
sns.despine(left=False, bottom=True)

plt.tight_layout()
plt.savefig("comparativa_modelos_grouped_bar.png", dpi=150, bbox_inches="tight")
plt.show()
print("Guardado: comparativa_modelos_grouped_bar.png")

```



Guardado: comparativa_modelos_grouped_bar.png

<h2 id="6b.-An%C3%A1lisis-de-discrepancia:-chrF++-Modelo-2-vs-Modelo-4">6b.

Análisis de discrepancia: chrF++ Modelo 2 vs Modelo 4</h2><p>El Modelo 2 supera a

Modelo 4 en chrF++ (43.4 vs 41.9) pero no en ninguna otra métrica. Esta sección

identifica los ejemplos específicos donde ocurre esa inversión y analiza por qué chrF++ y

Concept F1 divergen.</p>

```

In [48]: # — Análisis de discrepancia chrF++ M2 vs M4 —————
from sacrebleu.metrics import CHRF
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from scipy.stats import pearsonr

_chrf = CHRF(word_order=2)

# — chrF++ per-sample para M2 y M4 —————
def chrf_per_sample(model_dict, ref_dict, ids):
    scores = {}
    for sid in ids:

```



```

        hyp = " ".join(clean_sequence(model_dict[sid]["output"]))
        ref = " ".join(clean_sequence(ref_dict[sid]["traduccion"]))
        scores[sid] = _chrf.sentence_score(hyp, [ref]).score
    return scores

chrf_m2 = chrf_per_sample(model2_by_id, test_by_id, EVAL_IDS)
chrf_m4 = chrf_per_sample(model4_by_id, test_by_id, EVAL_IDS)

# — DataFrame comparativo —————
disc_rows = []
for sid in EVAL_IDS:
    s2, s4 = chrf_m2[sid], chrf_m4[sid]
    f1_m2 = per_sample["Modelo 2"].loc[sid, "Concept_F1"] if sid in per_sample[
    f1_m4 = per_sample["Modelo 4"].loc[sid, "Concept_F1"] if sid in per_sample[
    disc_rows.append({
        "id": sid,
        "chrF++_M2": round(s2, 2),
        "chrF++_M4": round(s4, 2),
        "delta_chrf": round(s2 - s4, 2),
        "ConceptF1_M2": round(f1_m2, 2),
        "ConceptF1_M4": round(f1_m4, 2),
        "delta_f1": round(f1_m2 - f1_m4, 2),
        "oracion": test_by_id[sid]["oracion"],
        "output_M2": model2_by_id[sid]["output"],
        "output_M4": model4_by_id[sid]["output"],
        "ref": test_by_id[sid]["traduccion"],
    })

df_disc = pd.DataFrame(disc_rows).set_index("id")
df_disc["inversion"] = (df_disc["delta_chrf"] > 0) & (df_disc["delta_f1"] <= 0)

print(f"Ejemplos donde M2 supera a M4 en chrF++: {(df_disc['delta_chrf'] > 0).su
print(f"De esos, con inversión en ConceptF1 (M4 >= M2): {(df_disc['inversion']).su

# — Top 10 —————
df_top = df_disc[df_disc["delta_chrf"] > 0].sort_values("delta_chrf", ascending=
cols = ["chrF++_M2", "chrF++_M4", "delta_chrf", "ConceptF1_M2", "ConceptF1_M4", "or
print("\n— Top 10: mayor ventaja de M2 sobre M4 en chrF++ —")
print(df_top[cols].head(10).to_string())

# — Análisis cualitativo top 3 —————
print("\n— Análisis cualitativo – top 3 casos —————")
for sid, row in df_top.head(3).iterrows():
    ref_toks = set(clean_sequence(row["ref"]))
    m2_toks = set(clean_sequence(row["output_M2"]))
    m4_toks = set(clean_sequence(row["output_M4"]))
    print(f"\nID {sid}")
    print(f" Oración : {row['oracion']}")
    print(f" Referencia: {row['ref']}")
    print(f" M2 output : {row['output_M2']}")
    print(f" M4 output : {row['output_M4']}")
    print(f" chrF++ → M2={row['chrF++_M2']:.1f} M4={row['chrF++_M4']:.1f} Δ=
    print(f" ConceptF1 → M2={row['ConceptF1_M2']:.1f} M4={row['ConceptF1_M4']:.
    print(f" M2 n Ref : {m2_toks & ref_toks} | M2 ruido: {m2_toks - ref_toks
    print(f" M4 n Ref : {m4_toks & ref_toks} | M4 ruido: {m4_toks - ref_toks
    print(f" — * 60)

# — Visualización —————
fig, axes = plt.subplots(1, 2, figsize=(13, 5))

```

```

df_clean = df_disc.dropna(subset=["ConceptF1_M2", "ConceptF1_M4"])
delta_c = df_clean["delta_chrf"]
delta_f = df_clean["delta_f1"]

ax = axes[0]
sc = ax.scatter(delta_c, delta_f, c=delta_c, cmap="RdYlGn",
                alpha=0.75, edgecolors="white", s=70, vmin=-40, vmax=40)
ax.axhline(0, color="gray", linestyle="--", alpha=0.5)
ax.axvline(0, color="gray", linestyle="--", alpha=0.5)
ax.set_xlabel("Δ chrF++ (M2 - M4)", fontsize=11)
ax.set_ylabel("Δ Concept F1 (M2 - M4)", fontsize=11)
ax.set_title("Discrepancia chrF++ vs Concept F1\npor ejemplo (M2 - M4)", fontwei
fig.colorbar(sc, ax=ax, label="Δ chrF++")
ax.text(0.97, 0.97, "M2 mejor en ambas", transform=ax.transAxes,
        ha="right", va="top", fontsize=8, color="green")
ax.text(0.03, 0.97, "Δ M2 mejor chrF++\nM4 mejor ConceptF1",
        transform=ax.transAxes, ha="left", va="top", fontsize=8, color="#c0392b")
ax.text(0.03, 0.03, "M4 mejor en ambas", transform=ax.transAxes,
        ha="left", va="bottom", fontsize=8, color="steelblue")

ax2 = axes[1]
ax2.hist(delta_c, bins=15, color="#3ecf8e", edgecolor="white", alpha=0.85)
ax2.axvline(0, color="black", linestyle="--", linewidth=1.5)
ax2.axvline(delta_c.mean(), color="#e74c3c", linestyle="-", linewidth=2,
            label=f"Media = {delta_c.mean():.2f}")
ax2.set_xlabel("Δ chrF++ (M2 - M4)", fontsize=11)
ax2.set_ylabel("Frecuencia", fontsize=11)
ax2.set_title("Distribución de Δ chrF++\n(> 0 significa M2 supera a M4)", fontwei
ax2.legend(fontsize=9)

plt.suptitle("Análisis de discrepancia: Modelo 2 vs Modelo 4 en chrF++",
            fontsize=13, fontweight="bold")
plt.tight_layout()
plt.savefig("discrepancia_chrf_m2_m4.png", dpi=150, bbox_inches="tight")
plt.show()
print("Guardado: discrepancia_chrf_m2_m4.png")

```

Ejemplos donde M2 supera a M4 en chrF++: 30 / 50
De esos, con inversión en ConceptF1 ($M4 \geq M2$): 15

— Top 10: mayor ventaja de M2 sobre M4 en chrF++ —

	chrF++_M2	chrF++_M4	delta_chrf	ConceptF1_M2	ConceptF1_M4	
oracion						
id						
961	100.00	59.21	40.79	100.00	0.00	
Preocupado						
951	58.98	24.98	34.00	57.14	0.00	
Cuando crezca, jugaremos juntos						
953	60.87	27.31	33.56	50.00	0.00	
Mis fortalezas						
18	52.71	26.40	26.31	40.00	0.00	
A ellos les gustaba jugar y cantar						
10	49.93	24.90	25.02	28.57	0.00	Siempre iba en
silla de ruedas, la paseaban sus abuelos						
962	58.28	33.99	24.29	80.00	40.00	
Partes del cartel						
957	45.63	21.69	23.94	40.00	0.00	
Mi Proyecto de vida						
963	66.64	43.25	23.39	57.14	40.00	
Es una piedra corazón						
2	77.76	56.34	21.43	66.67	40.00	Yo qu
iero usar la polea para mover los objetos						
8	61.83	41.21	20.62	60.00	22.22	
Esa fábrica producía cosas sin parar						

== Análisis cualitativo – top 3 casos ==

ID 961

Oración : Preocupado
Referencia: pict_26985
M2 output : pict_26985
M4 output : pict_7310 pict_26986
chrF++ → M2=100.0 M4=59.2 Δ=40.8
ConceptF1 → M2=100.0 M4=0.0
M2 n Ref : {'pict_26985'} | M2 ruido: set()
M4 n Ref : set() | M4 ruido: {'pict_26986', 'pict_7310'}

ID 951

Oración : Cuando crezca, jugaremos juntos
Referencia: pict_24053 pict_11683 pict_2439
M2 output : pict_24053 pict_7002 pict_2439 pict_5367
M4 output : pict_7310 pict_6625 pict_23991 pict_37952 pict_38937
chrF++ → M2=59.0 M4=25.0 Δ=34.0
ConceptF1 → M2=57.1 M4=0.0
M2 n Ref : {'pict_2439', 'pict_24053'} | M2 ruido: {'pict_5367', 'pict_7002'}
M4 n Ref : set() | M4 ruido: {'pict_23991', 'pict_6625', 'pict_37952', 'pict_38937', 'pict_7310'}

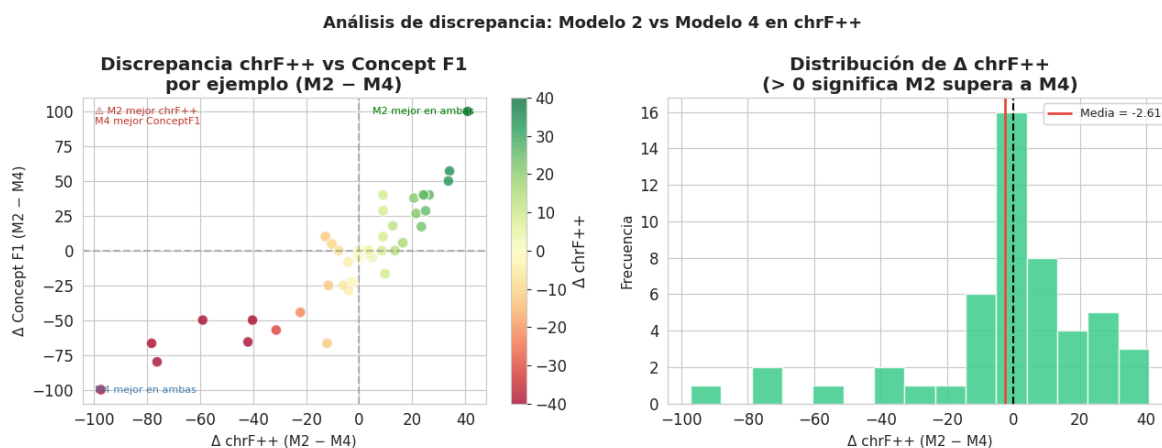
ID 953

Oración : Mis fortalezas
Referencia: pict_12266 pict_36539
M2 output : pict_12266 pict_8138
M4 output : pict_7310 pict_37621
chrF++ → M2=60.9 M4=27.3 Δ=33.6

ConceptF1 → M2=50.0 M4=0.0

M2 n Ref : {'pict_12266'} | M2 ruido: {'pict_8138'}

M4 n Ref : set() | M4 ruido: {'pict_37621', 'pict_7310'}



Guardado: discrepancia_chrf_m2_m4.png

<h2 id="Conclusiones:-Discrepancia-chrF++-Modelo-2-vs-Modelo-4">Conclusiones:

Discrepancia chrF++ Modelo 2 vs Modelo 4</h2><p>De los 50 ejemplos evaluados, **30 casos** (60%) muestran que Modelo 2 supera a Modelo 4 en chrF++, pero en **15 de esos casos** la situación se invierte en Concept F1 ($M4 \geq M2$).</p> <p>**¿Por qué ocurre esto?**</p>

- <p>**chrF++** mide similitud a nivel de caracteres y n-gramas de palabras. Si Modelo 2 genera tokens con IDs numéricamente similares a la referencia (mismas raíces morfológicas o números cercanos), obtiene un chrF++ alto aunque los tokens no sean exactamente los correctos.</p>
- <p>**Concept F1** exige coincidencia exacta de pict_IDs. Cualquier token no idéntico a la referencia es penalizado directamente.</p>

<p>**Caso más extremo (ID 961 — "Preocupado"):**</p>

- Modelo 2 genera `pict_26985` → coincide exactamente → chrF++=100, ConceptF1=100
- Modelo 4 genera `pict_7310 pict_26986` → ninguno coincide → chrF++=59.2, ConceptF1=0

<p>**Conclusión:** La discrepancia es coherente con la naturaleza de cada métrica. No indica un error del sistema sino que ambas métricas capturan aspectos distintos y complementarios de la calidad de predicción. chrF++ es más tolerante con variaciones léxicas; Concept F1 es más estricto semánticamente.</p>

<h1 id="7.-Análisis-de-concordancia-entre-evaluadores">7. Análisis de

concordancia entre evaluadores</h1><p>Usamos Krippendorff's Alpha para medir el acuerdo entre los 8 evaluadores humanos.</p>

- $\alpha > 0.8$: acuerdo fuerte
- $\alpha 0.6-0.8$: acuerdo moderado
- $\alpha < 0.6$: acuerdo débil

```
In [49]: # — Construir matriz de evaluaciones humanas —————
# Estructura: para cada (sentence_id, model), tenemos 8 scores

def parse_human_scores(evaluations, model_key="A_fiable"):
    """
    Extrae los scores de un modelo específico de todos los evaluadores.
    model_key: 'A_fiable', 'B_fiable', 'C_fiable', 'D_fiable'
    Retorna DataFrame: filas=evaluadores, columnas=sentence_ids
    """
    data = {}

    for ev in evaluations:
        user = ev["usuario"]
        scores = {}

        for resp in ev["respuestas"]:
            sid = resp["id"]
            ev_dict = resp["evaluacion"]

            if model_key in ev_dict:
                scores[sid] = ev_dict[model_key]

        data[user] = scores

    # filas = evaluadores, columnas = sentence_ids
    df = pd.DataFrame(data).T
    return df

# — Matriz para cada modelo —————
human_scores = {}

for mk, label in [
    ("A_fiable", "Modelo 1"),
    ("B_fiable", "Modelo 2"),
    ("C_fiable", "Modelo 3"),
    ("D_fiable", "Modelo 4")
]:
    human_scores[label] = parse_human_scores(evaluations, mk)

# — Vista general —————
print("=== Scores humanos - Modelo 1 (primeras 5 frases, primeros 3 evaluadores)")

sample_df = human_scores["Modelo 1"].iloc[:3, :5]

print(sample_df.to_string())
print(f"\nDimensiones: {human_scores['Modelo 1'].shape}")

=== Scores humanos - Modelo 1 (primeras 5 frases, primeros 3 evaluadores) ===
      0    1    2    3    4
a.paredes.um@gmail.com  3.0  5.0  4.0  3.0  5.0
angela.mariategui@gmail.com  NaN  NaN  NaN  NaN  NaN
CUSIPAUCARALACANTARAGMAIL.COM  3.0  4.0  4.0  4.0  4.0

Dimensiones: (8, 50)
```

```
In [50]: # — Krippendorff's Alpha —————
import krippendorff
alpha_rows = []
```

```

for label, df_h in human_scores.items():
    # krippendorff espera: rows=raters, cols=items (con NaN si falta)
    reliability_data = df_h.values.astype(float)
    alpha = krippendorff.alpha(reliability_data, level_of_measurement="ordinal")
    alpha_rows.append({"Modelo": label, "Krippendorff  $\alpha$ ": round(alpha, 4)})

df_alpha = pd.DataFrame(alpha_rows)
print("=== Acuerdo inter-anotador (Krippendorff  $\alpha$  ordinal) ===")
print(df_alpha.to_string(index=False))

# Interpretación
for _, row in df_alpha.iterrows():
    a = row["Krippendorff  $\alpha$ "]
    if a >= 0.8: interp = "Fuerte"
    elif a >= 0.6: interp = "Moderado"
    else: interp = "Débil"
    print(f" {row['Modelo']}: {interp} ( $\alpha$ ={{a}})")

```

```

=== Acuerdo inter-anotador (Krippendorff  $\alpha$  ordinal) ===
  Modelo  Krippendorff  $\alpha$ 
Modelo 1      0.1136
Modelo 2      0.3598
Modelo 3      0.2208
Modelo 4      0.1661
Modelo 1: Débil ( $\alpha$ =0.1136)
Modelo 2: Débil ( $\alpha$ =0.3598)
Modelo 3: Débil ( $\alpha$ =0.2208)
Modelo 4: Débil ( $\alpha$ =0.1661)

```

```

In [51]: # — Score humano promedio por modelo —————
human_mean_rows = []

for label, df_h in human_scores.items():
    mean_score = df_h.values.flatten()
    mean_score = mean_score[~np.isnan(mean_score)]

    human_mean_rows.append({
        "Modelo": label,
        "Score humano  $\mu$ ": round(np.mean(mean_score), 3),
        "Score humano  $\sigma$ ": round(np.std(mean_score), 3),
    })

df_human_mean = pd.DataFrame(human_mean_rows)

print("=== Scores humanos promedio ===")
print(df_human_mean.to_string(index=False))

# — Boxplot —————
fig, ax = plt.subplots(figsize=(10, 5))

plot_data = []
plot_labels = []

for label, df_h in human_scores.items():
    vals = df_h.values.flatten()
    vals = vals[~np.isnan(vals)]

    plot_data.append(vals)
    plot_labels.append(label)

```

```

bp = ax.boxplot(plot_data, labels=plot_labels, patch_artist=True)

colors_bp = sns.color_palette("husl", 4)

for patch, color in zip(bp["boxes"], colors_bp):
    patch.set_facecolor(color)
    patch.set_alpha(0.75)

ax.set_title(
    "Distribución de scores humanos por modelo (escala Likert 1-5)",
    fontweight="bold"
)

ax.set_ylabel("Score Humano")
ax.set_xlabel("Modelo")
ax.set_ylim(0.5, 5.5)

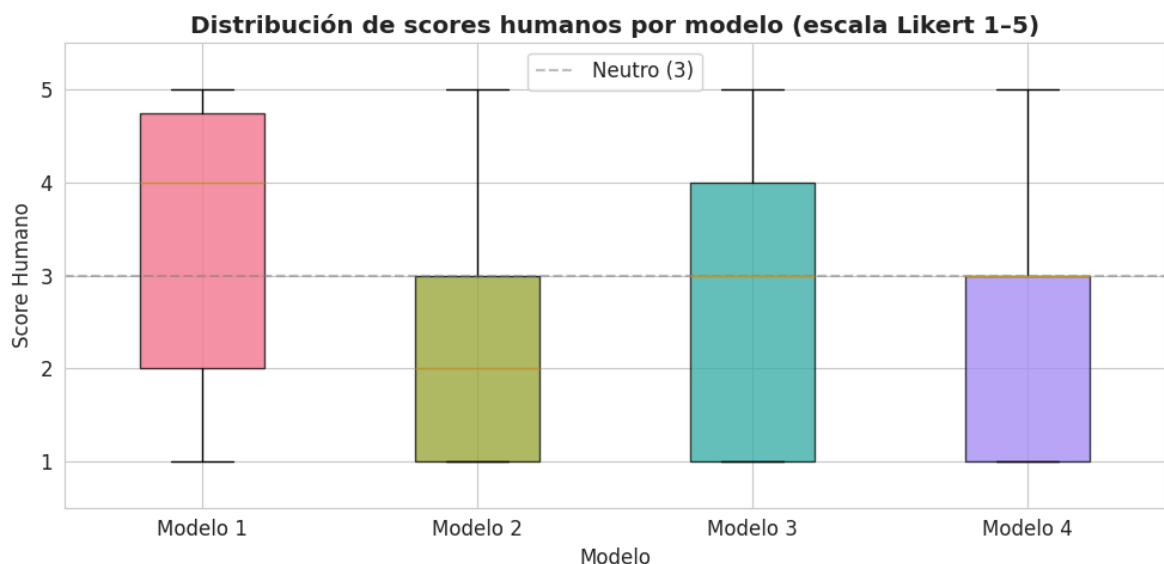
ax.axhline(3, color="gray", linestyle="--", alpha=0.5, label="Neutro (3)")
ax.legend()

plt.tight_layout()
plt.savefig("boxplot_scores_humanos.png", dpi=150, bbox_inches="tight")
plt.show()

```

=== Scores humanos promedio ===

Modelo	Score humano μ	Score humano σ
Modelo 1	3.251	1.460
Modelo 2	2.277	1.446
Modelo 3	2.940	1.510
Modelo 4	2.529	1.431



```

In [52]: # — Concordancia: ¿Cuándo Los evaluadores NO están de acuerdo? —
print("=== Análisis de desacuerdo (varianza por frase) ===\n")
disagreement_rows = []
for label, df_h in human_scores.items():
    variances = df_h.var(axis=0) # varianza entre evaluadores por frase
    high_var = variances[variances > 1.5].sort_values(ascending=False)
    disagreement_rows.append({
        "Modelo": label,
        "Frases con alta varianza (>1.5)": len(high_var),
        "Varianza media": round(variances.mean(), 3),
        "ID mayor desacuerdo": high_var.index[0] if len(high_var) > 0 else "-",
    })

```

```
})
```

```
df_disagree = pd.DataFrame(disagreement_rows)
print(df_disagree.to_string(index=False))
```

=== Análisis de desacuerdo (varianza por frase) ===

Modelo	Frases con alta varianza (>1.5)	Varianza media	ID mayor desacuerdo
Modelo 1	34	1.931	16
Modelo 2	19	1.315	16
Modelo 3	30	1.810	11
Modelo 4	28	1.713	9

<h1 id="8.-Correlaci%C3%B3n:-m%C3%A9tricas-autom%C3%A1ticas-vs.-evaluaci%C3%B3n-humana">8. Correlación: métricas automáticas vs. evaluación humana</h1><p>Calculamos correlación de Pearson (lineal) y Spearman (rango) entre cada métrica automática y el score humano promedio.</p>

```
In [53]: # — Construir DataFrame per-sample para correlación —————
def compute_per_sample_metrics(model_dict, ref_dict, ids):
    """Devuelve DataFrame con una fila por ejemplo evaluado."""
    rows = []
    bleu_s = BLEU(effective_order=True)
    chrf_s = CHRF(word_order=2)

    for sid in ids:
        oracion = ref_dict[sid]["oracion"]
        hyp_tokens = clean_sequence(model_dict[sid]["output"])
        ref_tokens = clean_sequence(ref_dict[sid]["traduccion"])
        hyp_str = " ".join(hyp_tokens)
        ref_str = " ".join(ref_tokens)
        hyp_text = tokens_to_text(hyp_tokens)

        # BLEU (sentence-level)
        bleu_val = bleu_s.sentence_score(hyp_str, [ref_str]).score
        # chrF++
        chrf_val = chrf_s.sentence_score(hyp_str, [ref_str]).score
        # Concept F1
        p, r, f1 = concept_precision_recall_f1(hyp_tokens, ref_tokens)
        # Coverage
        cov = coverage_score(hyp_tokens, ref_tokens)
        # Semantic Similarity
        sem_sim = get_semantic_similarity(oracion, hyp_text)
        rows.append({
            "id": sid,
            "BLEU": bleu_val,
            "chrF++": chrf_val,
            "Concept_F1": f1 * 100,
            "Coverage": cov * 100,
            "Semantic Similarity": sem_sim
        })
    return pd.DataFrame(rows).set_index("id")

# Construir tablas per-sample para cada modelo
per_sample = {
    "Modelo 1": compute_per_sample_metrics(model1_by_id, test_by_id, EVAL_IDS),
    "Modelo 2": compute_per_sample_metrics(model2_by_id, test_by_id, EVAL_IDS),
    "Modelo 3": compute_per_sample_metrics(model3_by_id, test_by_id, EVAL_IDS),
    "Modelo 4": compute_per_sample_metrics(model4_by_id, test_by_id, EVAL_IDS),
}
```



```
}

print("Métricas per-sample calculadas")
print(per_sample["Modelo 1"].head())
```

```
Métricas per-sample calculadas
      BLEU      chrF++  Concept_F1  Coverage  Semantic Similarity
id
0    9.578464  25.046951    0.000000    0.000000          99.126434
1   11.868405  22.664027    0.000000    0.000000          49.280178
2   62.444517  77.764304   66.666667   80.000000          77.010803
3   26.658377  55.150689   50.000000   66.666667          90.627411
4   55.032121  39.858717    0.000000    0.000000          72.265091
```

```
In [54]: # — Score humano promedio per-sample —————

def human_mean_per_sample(df_human):
    """Score medio entre evaluadores para cada frase."""
    return df_human.mean(axis=0) # serie indexada por sentence_id

human_mean_sample = {
    label: human_mean_per_sample(df_h)
    for label, df_h in human_scores.items()
}

# — Calcular correlaciones —————
corr_rows = []
for model_label in ["Modelo 1", "Modelo 2", "Modelo 3", "Modelo 4"]:
    metrics_df = per_sample[model_label]
    h_mean = human_mean_sample[model_label]

    # Alinear por id
    common_ids = metrics_df.index.intersection(h_mean.index)
    h_vals = h_mean.loc[common_ids].values

    for metric_col in ["BLEU", "chrF++", "Concept_F1", "Coverage", "Semantic Sim
        m_vals = metrics_df.loc[common_ids, metric_col].values

    # Quitar NaN
    mask = ~(np.isnan(m_vals) | np.isnan(h_vals))
    m_clean = m_vals[mask]
    h_clean = h_vals[mask]

    if len(m_clean) < 3:
        continue

    pearson_r, pearson_p = pearsonr(m_clean, h_clean)
    spearman_r, spearman_p = spearmanr(m_clean, h_clean)

    corr_rows.append({
        "Modelo": model_label,
        "Métrica": metric_col,
        "Pearson r": round(pearson_r, 4),
        "Pearson p": round(pearson_p, 4),
        "Spearman r": round(spearman_r, 4),
        "Spearman p": round(spearman_p, 4),
        "N": int(mask.sum()),
    })

df_corr = pd.DataFrame(corr_rows)
```

```
print("=== Correlación: métricas automáticas vs. score humano ===")
print(df_corr.to_string(index=False))
```

```
=== Correlación: métricas automáticas vs. score humano ===
```

Modelo	Métrica	Pearson r	Pearson p	Spearman r	Spearman p	N
Modelo 1	BLEU	0.2078	0.1476	0.1594	0.2690	50
Modelo 1	chrF++	0.1277	0.3770	0.0917	0.5266	50
Modelo 1	Concept_F1	0.0729	0.6151	0.0334	0.8181	50
Modelo 1	Coverage	0.0827	0.5680	0.0837	0.5635	50
Modelo 1	Semantic Similarity	0.3454	0.0140	0.2279	0.1114	50
Modelo 2	BLEU	0.2536	0.0755	0.2363	0.0985	50
Modelo 2	chrF++	0.1501	0.2982	0.1102	0.4462	50
Modelo 2	Concept_F1	0.3404	0.0156	0.3196	0.0237	50
Modelo 2	Coverage	0.3085	0.0293	0.2915	0.0400	50
Modelo 2	Semantic Similarity	0.5005	0.0002	0.5707	0.0000	50
Modelo 3	BLEU	0.2691	0.0588	0.1710	0.2352	50
Modelo 3	chrF++	0.2059	0.1514	0.0543	0.7079	50
Modelo 3	Concept_F1	0.1938	0.1774	0.0687	0.6356	50
Modelo 3	Coverage	0.1436	0.3198	0.0292	0.8405	50
Modelo 3	Semantic Similarity	0.0575	0.6914	0.1037	0.4735	50
Modelo 4	BLEU	0.0066	0.9637	0.0449	0.7568	50
Modelo 4	chrF++	0.0072	0.9604	-0.0033	0.9819	50
Modelo 4	Concept_F1	-0.0884	0.5418	-0.0878	0.5442	50
Modelo 4	Coverage	0.0121	0.9335	0.0001	0.9993	50
Modelo 4	Semantic Similarity	0.2660	0.0618	0.4221	0.0023	50

```
In [55]: # — Heatmap de correlaciones —————
metric_order = ["BLEU", "chrF++", "Concept_F1", "Coverage", "Semantic Similarity"]

df_corr["Métrica"] = pd.Categorical(
    df_corr["Métrica"],
    categories=metric_order,
    ordered=True
)

fig, axes = plt.subplots(1, 2, figsize=(16, 8))

for ax, corr_col, title in [
    (axes[0], "Pearson r", "Correlación de Pearson (r)"),
    (axes[1], "Spearman r", "Correlación de Spearman (p)"),
]:
    pivot = df_corr.pivot(index="Modelo", columns="Métrica", values=corr_col)

    sns.heatmap(
        pivot,
        annot=True,
        fmt=".3f",
        cmap="RdYlGn",
        center=0,
        vmin=-1,
        vmax=1,
        linewidths=0.5,
        ax=ax,
        cbar_kws={"label": corr_col}
    )

    ax.set_title(title, fontweight="bold")

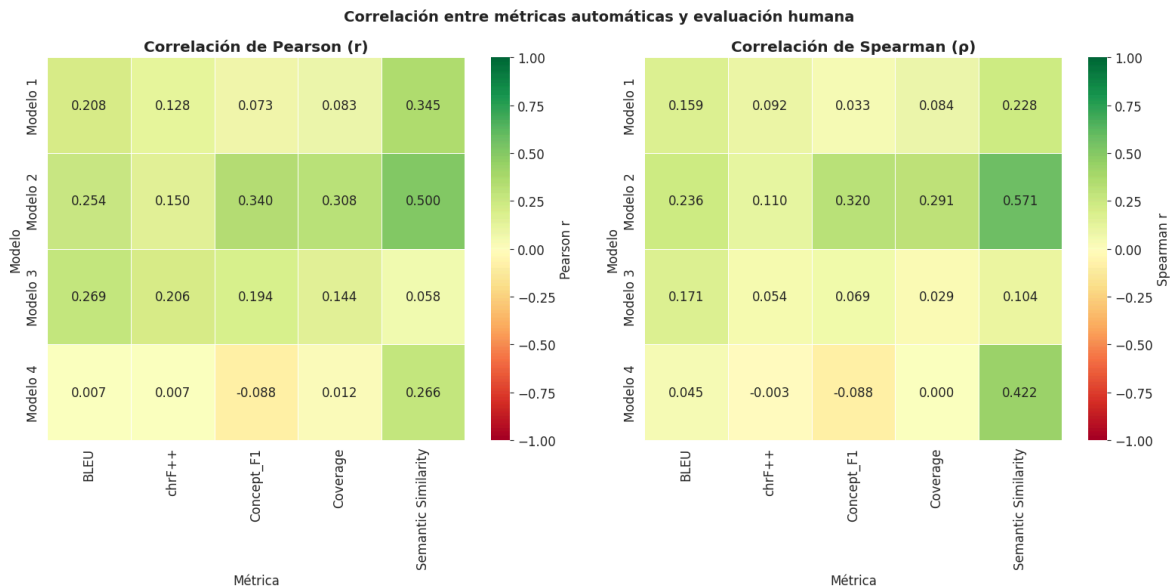
plt.suptitle(
```

```

    "Correlación entre métricas automáticas y evaluación humana",
    fontsize=14,
    fontweight="bold"
)

plt.tight_layout()
plt.savefig("correlacion_metricas.png", dpi=150, bbox_inches="tight")
plt.show()

```



```

In [56]: # — Scatter plots: mejor vs peor métrica correlacionada —————
# (usa el modelo con más varianza para visualización)

target_model = "Modelo 1"
df_m = per_sample[target_model]
h_m = human_mean_sample[target_model]
common = df_m.index.intersection(h_m.index)

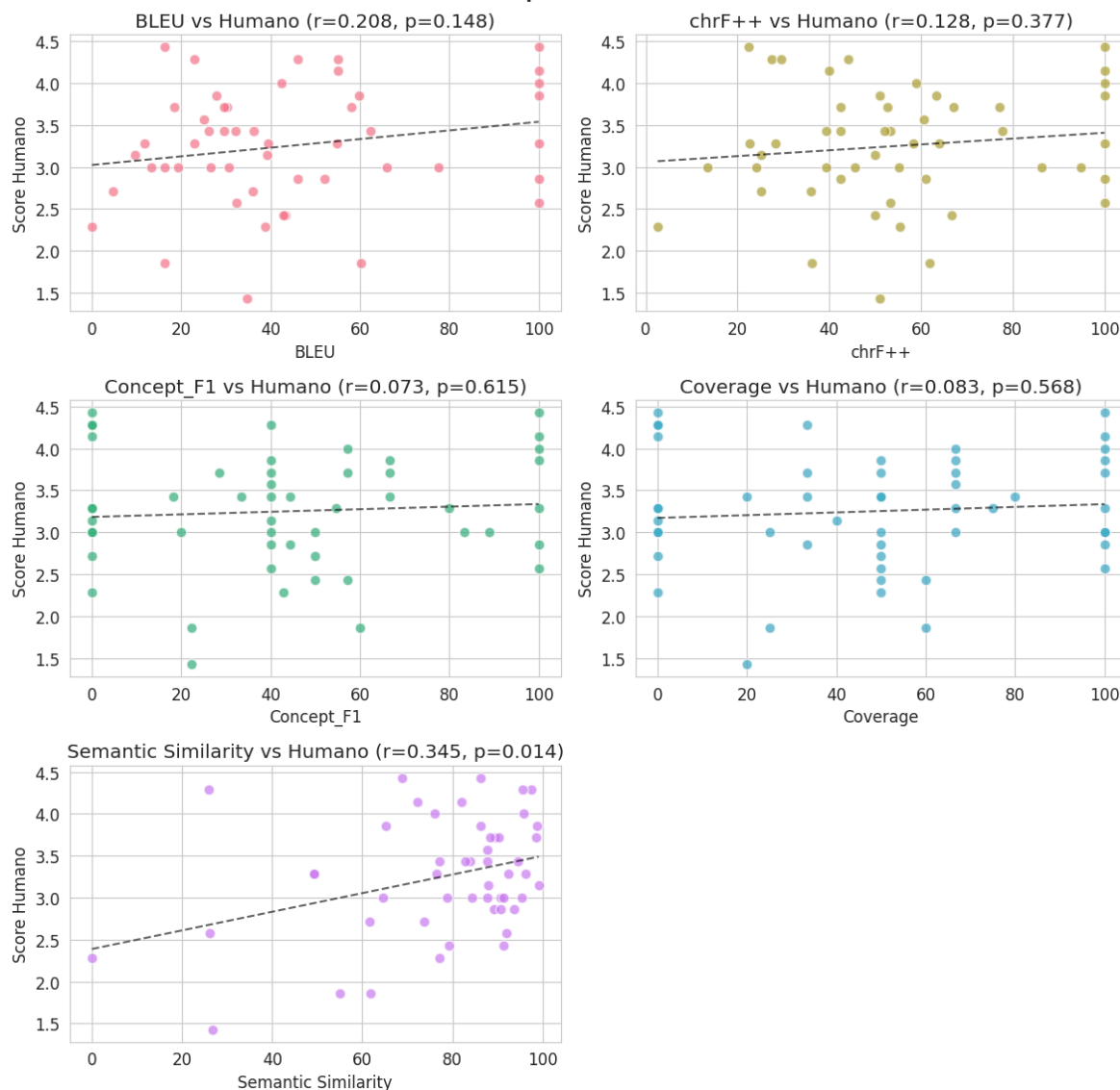
fig, axes = plt.subplots(3, 2, figsize=(12, 12))
metrics = ["BLEU", "chrF++", "Concept_F1", "Coverage", "Semantic Similarity"]
colors_sc = sns.color_palette("husl", 5)

for ax, metric, color in zip(axes.flat, metrics, colors_sc):
    x = df_m.loc[common, metric].values
    y = h_m.loc[common].values
    mask = ~(np.isnan(x) | np.isnan(y))
    r, p = pearsonr(x[mask], y[mask])
    ax.scatter(x[mask], y[mask], color=color, alpha=0.7, edgecolors="white", s=6)
    # Línea de tendencia
    z = np.polyfit(x[mask], y[mask], 1)
    pf = np.poly1d(z)
    xs = np.linspace(x[mask].min(), x[mask].max(), 100)
    ax.plot(xs, pf(xs), "k--", linewidth=1.5, alpha=0.6)
    ax.set_xlabel(metric)
    ax.set_ylabel("Score Humano")
    ax.set_title(f"{metric} vs Humano (r={r:.3f}, p={p:.3f})")

axes.flat[-1].axis('off')
plt.suptitle(f"Scatter plots - {target_model}", fontsize=14, fontweight="bold")
plt.tight_layout()
plt.savefig("scatter_correlacion.png", dpi=150, bbox_inches="tight")
plt.show()

```

Scatter plots - Modelo 1



<h2 id="8b.-An%C3%A1lisis-de-discrepancia:-Semantic-Similarity-alto-vs-metricas-semanticas-bajas-Modelo-4">8b. Análisis de discrepancia: Semantic Similarity alto vs metricas semanticas bajas Modelo 4</h2>

```
In [75]: # — Análisis de discrepancia:
# Semantic Similarity alto vs overlap bajo (Modelo 4)
# —————

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

# — Similaridad semántica per-sample —————

def semantic_similarity_per_sample(model_dict, ref_dict, ids):
    scores = {}

    for sid in ids:

        # texto original real
        original_text = ref_dict[sid]["oracion"]

        # salida del modelo → keywords reales
        hyp_tokens = clean_sequence(model_dict[sid]["output"])
```

```

        hyp_text = tokens_to_text(hyp_tokens)

        sim = get_semantic_similarity(original_text, hyp_text)

        scores[sid] = sim

    return scores

sem_m4 = semantic_similarity_per_sample(
    model4_by_id,
    test_by_id,
    EVAL_IDS
)

# — Construcción DataFrame —————

rows = []

for sid in EVAL_IDS:

    rows.append({
        "id": sid,
        "BLEU": per_sample["Modelo 4"].loc[sid, "BLEU"],
        "chrF++": per_sample["Modelo 4"].loc[sid, "chrF++"],
        "Concept_F1": per_sample["Modelo 4"].loc[sid, "Concept_F1"],
        "Coverage": per_sample["Modelo 4"].loc[sid, "Coverage"],
        "Semantic_Similarity": sem_m4[sid],
        "Human": human_mean_sample["Modelo 4"].loc[sid],
        "oracion": test_by_id[sid]["oracion"],
        "ref": test_by_id[sid]["traduccion"],
        "output": model4_by_id[sid]["output"],
    })

df_sem_disc = pd.DataFrame(rows).set_index("id")

# — Casos interesantes —————
# Semantic Similarity alta
# pero overlap bajo

interesting = df_sem_disc[
    (df_sem_disc["Semantic_Similarity"] >= 70)
    &
    (
        (df_sem_disc["BLEU"] < 20)
        |
        (df_sem_disc["Concept_F1"] < 30)
        |
        (df_sem_disc["Coverage"] < 30)
    )
].sort_values(
    "Semantic_Similarity",
    ascending=False
)

print(f"Casos encontrados: {len(interesting)}")

# — Top casos —————

cols = [
    "Semantic_Similarity",

```

```

    "BLEU",
    "chrF++",
    "Concept_F1",
    "Coverage",
    "Human",
    "oracion"
]

print("\n== Top casos: alta similitud semántica pero bajo overlap ==")
print(
    interesting[cols]
    .head(10)
    .round(2)
    .to_string()
)

```

Casos encontrados: 23

== Top casos: alta similitud semántica pero bajo overlap ==

	Semantic_Similarity	BLEU	chrF++	Concept_F1	Coverage	Human	
oracion							
id							
972	98.90	16.25	40.84	22.22	20.0	3.00	
¿Cuál es tu comida favorita?							
0	96.78	13.49	25.29	0.00	0.0	3.00	
Cerrar los ojos y dormir							
952	95.53	22.96	27.34	0.00	0.0	4.29	
Compartir con amigos							
7	95.17	11.73	28.27	0.00	0.0	2.71	
Esperar en fila tu turno							
5	93.20	8.12	32.23	0.00	0.0	2.71	Estoy tranq
uilo mientras la enfermera me mide la temperatura							
24	92.71	40.64	40.56	25.00	25.0	4.00	
El corazón mueve la sangre por el cuerpo							
14	92.31	17.75	64.11	50.00	100.0	2.86	
Dar abrazos							
11	91.93	16.19	35.86	22.22	25.0	3.14	
La doctora me mira los oídos con el otoscopio							
968	91.26	16.23	24.01	0.00	0.0	3.29	
Trabajos manuales							
970	89.30	20.86	34.85	25.00	25.0	3.14	
En la granja, les dieron comida a las gallinas							

```

In [85]: # — Análisis cualitativo enriquecido —————

print("\n== Análisis cualitativo - top 3 casos =====")

for sid, row in interesting.head(3).iterrows():

    ref_tokens = clean_sequence(row["ref"])
    out_tokens = clean_sequence(row["output"])

    ref_text = tokens_to_text(ref_tokens)
    out_text = tokens_to_text(out_tokens)

    print("\n" + "=" * 80)

    print(f"ID {sid}")

    # — Texto original —————

```

```
print(f"\nTexto original:")
print(row["oracion"])

# — Referencia —————
print(f"\nReferencia pictogramas:")
print(row["ref"])

print(f"\nReferencia traducida:")
print(ref_text)

# — Predicción —————
print(f"\nPredicción Modelo 4:")
print(row["output"])

print(f"\nPredicción traducida:")
print(out_text)

# — Métricas —————
print("\nMétricas:")

print(f"  Semantic Similarity : {row['Semantic_Similarity']:.2f}")
print(f"  BLEU                  : {row['BLEU']:.2f}")
print(f"  chrF++                 : {row['chrF++']:.2f}")
print(f"  Concept_F1             : {row['Concept_F1']:.2f}")
print(f"  Coverage               : {row['Coverage']:.2f}")
print(f"  Score humano           : {row['Human']:.2f}")

print("\n" * 70)
```

= Análisis cualitativo – top 3 casos

ID 972

Texto original:

¿Cuál es tu comida favorita?

Referencia pictogramas:

pict_22620 pict_5581 pict_6625 pict_4610 pict_30012

Referencia traducida:

qué ser tú comida favorito

Predicción Modelo 4:

pict_12281 pict_4611 pict_30012 pict_26022

Predicción traducida:

tu comida favorito ¿cuál?

Métricas:

Semantic Similarity	: 98.90
BLEU	: 16.25
chrF++	: 40.84
Concept_F1	: 22.22
Coverage	: 20.00
Score humano	: 3.00

ID 0

Texto original:

Cerrar los ojos y dormir

Referencia pictogramas:

pict_10146 pict_6479

Referencia traducida:

cerrar dormir

Predicción Modelo 4:

pict_7056 pict_2876 pict_28623

Predicción traducida:

cerrar ojos dormir

Métricas:

Semantic Similarity	: 96.78
BLEU	: 13.49
chrF++	: 25.29
Concept_F1	: 0.00
Coverage	: 0.00
Score humano	: 3.00

ID 952

Texto original:

Compartir con amigos

Referencia pictogramas:
pict_38900 pict_25792

Referencia traducida:
compartir amigos

Predicción Modelo 4:
pict_15361 pict_2255

Predicción traducida:
compartir amigos

Métricas:

Semantic Similarity	: 95.53
BLEU	: 22.96
chrF++	: 27.34
Concept_F1	: 0.00
Coverage	: 0.00
Score humano	: 4.29

```
In [88]: # — Scatter plot —————

fig, ax = plt.subplots(figsize=(7, 6))

x = df_sem_disc["Concept_F1"]
y = df_sem_disc["Semantic_Similarity"]

sc = ax.scatter(
    x,
    y,
    c=df_sem_disc["Human"],
    cmap="viridis",
    alpha=0.8,
    s=80,
    edgecolors="white"
)

ax.axvline(30, linestyle="--", alpha=0.5, color="gray")
ax.axhline(70, linestyle="--", alpha=0.5, color="gray")

ax.set_xlabel("Concept F1")
ax.set_ylabel("Semantic Similarity")

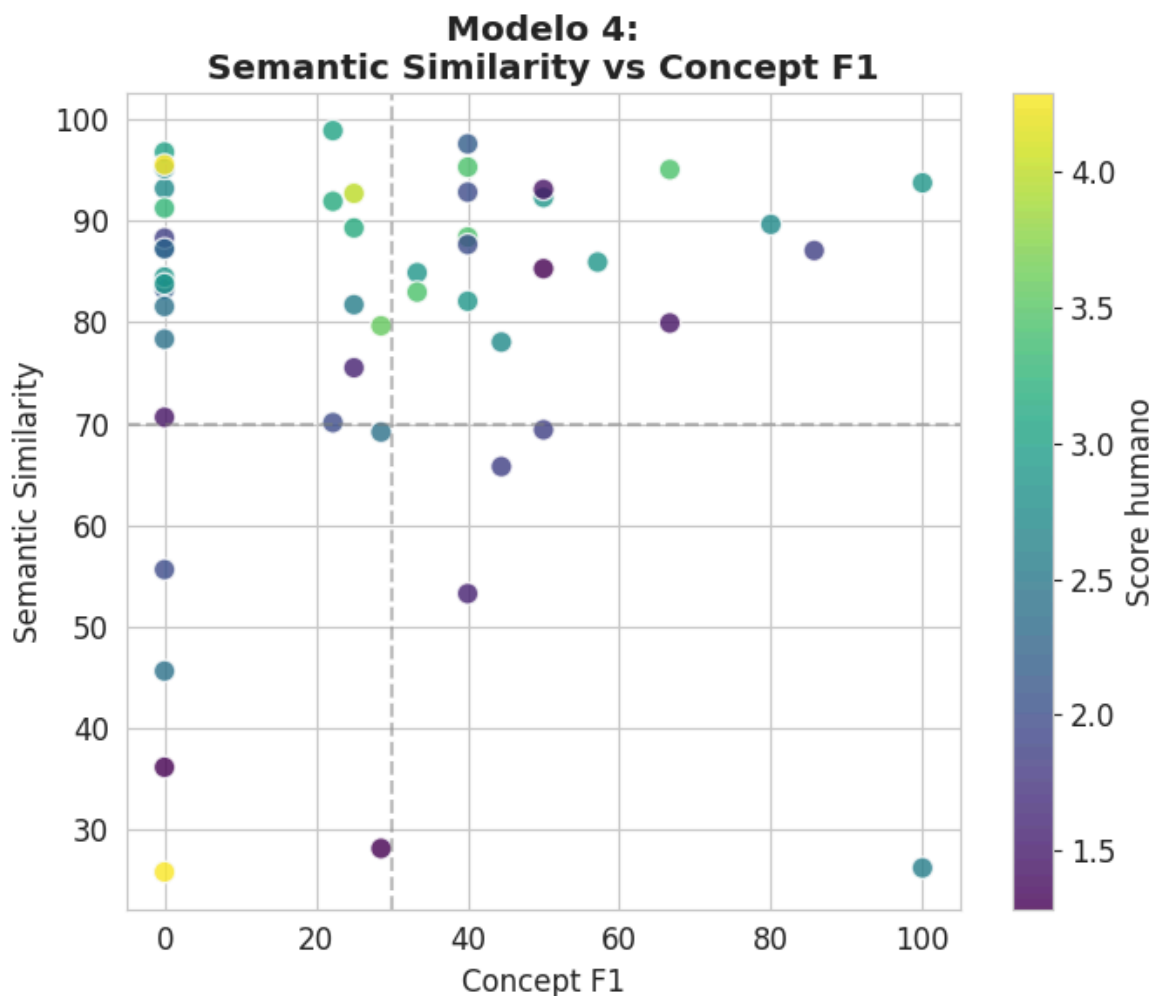
ax.set_title(
    "Modelo 4:\nSemantic Similarity vs Concept F1",
    fontweight="bold"
)

cbar = plt.colorbar(sc)
cbar.set_label("Score humano")

plt.tight_layout()

plt.savefig(
    "semantic_similarity_vs_overlap_m4.png",
    dpi=150,
    bbox_inches="tight"
```

```
)
plt.show()
print("Guardado: semantic_similarity_vs_overlap_m4.png")
```



Guardado: semantic_similarity_vs_overlap_m4.png

Conclusiones: Discrepancia Semantic Similarity vs métricas de overlap en Modelo 4

De los 50 ejemplos evaluados, se identificaron 23 casos donde el Modelo 4 obtuvo una alta Semantic Similarity (≥ 70) pese a presentar valores bajos en métricas basadas en coincidencia explícita como BLEU, Concept F1 o Coverage.

¿Por qué ocurre esto?

Semantic Similarity evalúa la cercanía semántica entre el texto original y la representación textual derivada de los pictogramas generados por el modelo. A diferencia de las demás métricas, esta comparación no utiliza la secuencia de referencia, sino directamente el texto original del dataset frente a la traducción textual de la predicción del modelo.

Por el contrario:

- BLEU y chrF++ dependen del solapamiento textual y del orden de los tokens respecto a la referencia.
- Concept F1 exige coincidencia exacta entre pictogramas.
- Coverage penaliza directamente los conceptos omitidos respecto a la secuencia esperada.

<h3 id="Caso-representativo-(ID-972-%E2%80%94-%E2%80%9C¿Cuál es tu comida favorita?%E2%80%9D)">Caso representativo (ID 972 — "¿Cuál es tu comida favorita?")</h3><h4 id="Texto-original">Texto original</h4>
¿Cuál es tu comida favorita?

Referencia

pict_22620 pict_5581 pict_6625 pict_4610 pict_30012

→ qué ser tú comida favorito

Predicción Modelo 4

pict_12281 pict_4611 pict_30012 pict_26022

→ tu comida favorito ¿cuál?

Aunque los pictogramas utilizados por el Modelo 4 son diferentes a los de referencia y el overlap es bajo (ConceptF1 = 22.22, Coverage = 20), la representación textual generada conserva prácticamente la misma intención comunicativa de la oración original.

Por esta razón:

- **Semantic Similarity** alcanza un valor extremadamente alto (**98.90**),
- mientras que BLEU, Concept F1 y Coverage penalizan la diferencia estructural respecto a la referencia.

<p>Este comportamiento evidencia que Semantic Similarity captura preservación del significado global e inteligibilidad contextual, incluso cuando la secuencia pictográfica no coincide explícitamente con la esperada.</p> <h3>

id="Conclusi%C3%B3n">Conclusión</h3><p>La discrepancia observada es coherente con la naturaleza de cada métrica y no representa un error del sistema. Semantic Similarity mide cercanía conceptual respecto al significado de la oración original, mientras que BLEU, chrF++, Concept F1 y Coverage evalúan fidelidad estructural respecto a la secuencia de referencia.</p><p>Los resultados sugieren que el Modelo 4 tiende a generar secuencias semánticamente plausibles pero estructuralmente diferentes a las esperadas, lo que refuerza la importancia de combinar métricas semánticas y métricas de overlap durante la evaluación de sistemas pictográficos.</p>

9. Implementación de LLM-Judge

<p>Celda 1: Instalación de la librería oficial</p> <p>Asegúrate de ejecutar esto primero para contar con el SDK moderno de Google Gen AI.</p>

```
In [57]: %pip install google-genai
```

Collecting google-genai

Downloading google_genai-2.6.0-py3-none-any.whl.metadata (52 kB)

Requirement already satisfied: anyio<5.0.0,>=4.8.0 in /home/dg/.local/share/pipx/venvs/notebook/lib/python3.12/site-packages (from google-genai) (4.13.0)

Collecting google-auth<3.0.0,>=2.48.1 (from google-auth[requests]<3.0.0,>=2.48.1->google-genai)

Downloading google_auth-2.53.0-py3-none-any.whl.metadata (5.5 kB)

Requirement already satisfied: httpx<1.0.0,>=0.28.1 in /home/dg/.local/share/pipx/venvs/notebook/lib/python3.12/site-packages (from google-genai) (0.28.1)

Collecting pydantic<3.0.0,>=2.9.0 (from google-genai)

Using cached pydantic-2.13.4-py3-none-any.whl.metadata (109 kB)

Requirement already satisfied: requests<3.0.0,>=2.28.1 in /home/dg/.local/share/pipx/venvs/notebook/lib/python3.12/site-packages (from google-genai) (2.34.2)

Collecting tenacity<9.2.0,>=8.2.3 (from google-genai)

Downloading tenacity-9.1.4-py3-none-any.whl.metadata (1.2 kB)

Collecting websockets<17.0,>=13.0.0 (from google-genai)

Downloading websockets-16.0-cp312-cp312-manylinux1_x86_64.manylinux_2_28_x86_64.manylinux_2_5_x86_64.whl.metadata (6.8 kB)

Requirement already satisfied: typing-extensions<5.0.0,>=4.14.0 in /home/dg/.local/share/pipx/venvs/notebook/lib/python3.12/site-packages (from google-genai) (4.15.0)

Collecting distro<2,>=1.7.0 (from google-genai)

Downloading distro-1.9.0-py3-none-any.whl.metadata (6.8 kB)

Collecting sniffio (from google-genai)

Downloading sniffio-1.3.1-py3-none-any.whl.metadata (3.9 kB)

Requirement already satisfied: idna>=2.8 in /home/dg/.local/share/pipx/venvs/notebook/lib/python3.12/site-packages (from anyio<5.0.0,>=4.8.0->google-genai) (3.16)

Collecting pyasn1-modules>=0.2.1 (from google-auth<3.0.0,>=2.48.1->google-auth[requests]<3.0.0,>=2.48.1->google-genai)

Downloading pyasn1_modules-0.4.2-py3-none-any.whl.metadata (3.5 kB)

Collecting cryptography>=38.0.3 (from google-auth<3.0.0,>=2.48.1->google-auth[requests]<3.0.0,>=2.48.1->google-genai)

Downloading cryptography-48.0.0-cp311-abi3-manylinux_2_34_x86_64.whl.metadata (4.3 kB)

Requirement already satisfied: certifi in /home/dg/.local/share/pipx/venvs/notebook/lib/python3.12/site-packages (from httpx<1.0.0,>=0.28.1->google-genai) (2026.5.20)

Requirement already satisfied: httpcore==1.* in /home/dg/.local/share/pipx/venvs/notebook/lib/python3.12/site-packages (from httpx<1.0.0,>=0.28.1->google-genai) (1.0.9)

Requirement already satisfied: h11>=0.16 in /home/dg/.local/share/pipx/venvs/notebook/lib/python3.12/site-packages (from httpcore==1.*->httpx<1.0.0,>=0.28.1->google-genai) (0.16.0)

Collecting annotated-types>=0.6.0 (from pydantic<3.0.0,>=2.9.0->google-genai)

Using cached annotated_types-0.7.0-py3-none-any.whl.metadata (15 kB)

Collecting pydantic-core==2.46.4 (from pydantic<3.0.0,>=2.9.0->google-genai)

Using cached pydantic_core-2.46.4-cp312-cp312-manylinux_2_17_x86_64.manylinux2014_x86_64.whl.metadata (6.6 kB)

Collecting typing-inspection>=0.4.2 (from pydantic<3.0.0,>=2.9.0->google-genai)

Using cached typing_inspection-0.4.2-py3-none-any.whl.metadata (2.6 kB)

Requirement already satisfied: charset-normalizer<4,>=2 in /home/dg/.local/share/pipx/venvs/notebook/lib/python3.12/site-packages (from requests<3.0.0,>=2.28.1->google-genai) (3.4.7)

Requirement already satisfied: urllib3<3,>=1.26 in /home/dg/.local/share/pipx/venvs/notebook/lib/python3.12/site-packages (from requests<3.0.0,>=2.28.1->google-genai) (2.7.0)

Requirement already satisfied: cffi>=2.0.0 in /home/dg/.local/share/pipx/venvs/notebook/lib/python3.12/site-packages (from cryptography>=38.0.3->google-auth<3.0.0,>=2.48.1->google-auth[requests]<3.0.0,>=2.48.1->google-genai) (2.0.0)

Requirement already satisfied: pycparser in /home/dg/.local/share/pipx/venvs/notebook/lib/python3.12/site-packages (from cffi>=2.0.0->cryptography>=38.0.3->google-auth<3.0.0,>=2.48.1->google-auth[requests]<3.0.0,>=2.48.1->google-genai) (2.0.0)

```

book/lib/python3.12/site-packages (from cffi>=2.0.0->cryptography>=38.0.3->google
-auth<3.0.0,>=2.48.1->google-auth[requests]<3.0.0,>=2.48.1->google-genai) (3.0)
Collecting pyasn1<0.7.0,>=0.6.1 (from pyasn1-modules>=0.2.1->google-auth<3.0.0,>=
2.48.1->google-auth[requests]<3.0.0,>=2.48.1->google-genai)
  Downloading pyasn1-0.6.3-py3-none-any.whl.metadata (8.4 kB)
  Downloading google_genai-2.6.0-py3-none-any.whl (821 kB)
      821.0/821.0 kB 5.9 MB/s 0:00:00
Downloading distro-1.9.0-py3-none-any.whl (20 kB)
Downloading google_auth-2.53.0-py3-none-any.whl (246 kB)
Using cached pydantic-2.13.4-py3-none-any.whl (472 kB)
Using cached pydantic_core-2.46.4-cp312-cp312-manylinux_2_17_x86_64.manylinux2014
_x86_64.whl (2.1 MB)
Downloading tenacity-9.1.4-py3-none-any.whl (28 kB)
Downloading websockets-16.0-cp312-cp312-manylinux1_x86_64.manylinux_2_28_x86_64.m
anylinux_2_5_x86_64.whl (184 kB)
Using cached annotated_types-0.7.0-py3-none-any.whl (13 kB)
Downloading cryptography-48.0.0-cp311-abi3-manylinux_2_34_x86_64.whl (4.7 MB)
      4.7/4.7 MB 27.4 MB/s 0:00:00
Downloading pyasn1_modules-0.4.2-py3-none-any.whl (181 kB)
Downloading pyasn1-0.6.3-py3-none-any.whl (83 kB)
Using cached typing_inspection-0.4.2-py3-none-any.whl (14 kB)
Downloading sniffio-1.3.1-py3-none-any.whl (10 kB)
Installing collected packages: websockets, typing-inspection, tenacity, sniffio,
pydantic-core, pyasn1, distro, annotated-types, pydantic, pyasn1-modules, cryptog
raphy, google-auth, google-genai
      13/13 [google-genai]— 12/13 [google
-genai]s]
Successfully installed annotated-types-0.7.0 cryptography-48.0.0 distro-1.9.0 goo
gle-auth-2.53.0 google-genai-2.6.0 pyasn1-0.6.3 pyasn1-modules-0.4.2 pydantic-2.1
3.4 pydantic-core-2.46.4 sniffio-1.3.1 tenacity-9.1.4 typing-inspection-0.4.2 web
sockets-16.0
Note: you may need to restart the kernel to use updated packages.

```

<p>Celda 2: Configuración del Evaluador Estructurado (LLM-Judge)</p> <p>En esta celda inicializamos el cliente de la API y definimos el esquema estricto de salida (response_schema) para que la IA devuelva exactamente las llaves solicitadas: score, semantic_errors, missing_concepts y comments.</p>

```

In [58]: import os
import json
import time
from pathlib import Path
from typing import Any

import pandas as pd
import requests

# --- CARGA ESTRUCTA DEL ARCHIVO .ENV ---
dotenv_path = Path('.env')

if dotenv_path.exists():
    print("✅ Archivo .env detectado con éxito.")
    for line in dotenv_path.read_text(encoding='utf-8').splitlines():
        line = line.strip()
        if not line or line.startswith('#') or '=' not in line:
            continue
        key, value = line.split('=', 1)
        os.environ[key.strip()] = value.strip()
    if "GEMINI_API_KEY" in os.environ:

```

```

print("✅ GEMINI_API_KEY cargada correctamente en el entorno de Python.")
else:
    print("❌ Error: El archivo .env existe, pero no contiene la línea 'GEMINI_API_KEY='")
else:
    print("❌ Error crítico: No se encontró el archivo '.env' en el directorio actual.")
    print(f"Ruta actual donde busca el notebook: {Path('.').resolve()}")

GEMINI_API_KEY = os.environ.get("GEMINI_API_KEY")
GEMINI_URL = "https://generativelanguage.googleapis.com/v1beta/models/gemini-flan-001:generateText?key=" + GEMINI_API_KEY

PROMPT_JUEZ_SISTEMA = """
Eres un experto lingüista y evaluador de sistemas de comunicación aumentativa y alternativa.
Tu tarea es evaluar qué tan bien una secuencia de pictogramas representa el significado de un texto.

Evalúa bajo los siguientes criterios (Escala del 1 al 5):
- 5 (Excelente): No falta ningún concepto, el orden es coherente y no hay errores.
- 4 (Bueno): Se entiende perfectamente, pero omitió algún elemento secundario o el orden no es perfecto.
- 3 (Aceptable): El sentido general se captura, pero faltan conceptos importantes.
- 2 (Deficiente): Hay errores semánticos graves; la secuencia confunde el mensaje.
- 1 (Inaceptable): No tiene ninguna relación semántica con el texto.

OBLIGATORIO: Debes responder ÚNICAMENTE con un objeto JSON válido que contenga la siguiente estructura:
{
  "score": [número entero del 1 al 5],
  "semantic_errors": [lista de strings con errores detectados],
  "missing_concepts": [lista de strings con conceptos del texto original faltantes],
  "comments": [string con tu justificación detallada de la nota]
}

No agregues texto introductorio ni explicaciones fuera del JSON.
"""

def _extract_json_from_response(text: str) -> dict[str, Any]:
    return json.loads(text)

def evaluar_con_llm_judge(texto_original, pictogramas_generados, prompt_sistema=
    """Llama a Gemini vía REST usando el endpoint recomendado por Google AI Studio""",
    if not GEMINI_API_KEY:
        return {
            "score": None,
            "semantic_errors": ["Falta GEMINI_API_KEY en el entorno"],
            "missing_concepts": [],
            "comments": "No se encontró la clave API en .env",
        }

    payload = {
        "systemInstruction": {"parts": [{"text": prompt_sistema}]},
        "contents": [
            {
                "parts": [
                    {
                        "text": f'Texto Original: "{texto_original}"\nSecuencia de Pictogramas: "{pictogramas_generados}"'
                    }
                ]
            }
        ],
        "generationConfig": {
            "temperature": 0.1,
            "responseMimeType": "application/json",
        }
    }

    headers = {"Content-Type": "application/json"}

    response = requests.post(GEMINI_URL, headers=headers, data=json.dumps(payload))

    if response.status_code == 200:
        return _extract_json_from_response(response.text)
    else:
        print(f"Error al llamar a Gemini: {response.status_code} - {response.text}")
        return {}

```

```

    },
}

response = requests.post(
    GEMINI_URL,
    headers={
        "Content-Type": "application/json",
        "X-goog-api-key": GEMINI_API_KEY,
    },
    json=payload,
    timeout=60,
)

if response.status_code == 429:
    return {
        "score": None,
        "semantic_errors": [f"Error de ejecución: 429 RESOURCE_EXHAUSTED. {r"},
        "missing_concepts": [],
        "comments": "Cuota agotada o límite de peticiones alcanzado.",
    }

response.raise_for_status()
response_data = response.json()
text = response_data["candidates"][0]["content"]["parts"][0]["text"]
return _extract_json_from_response(text)

```

✗ Error crítico: No se encontró el archivo '.env' en el directorio actual.
Ruta actual donde busca el notebook: /home/dg/Proyectos/DataSoftInteligente

<p>Distintos prompts para verificar consistencia: Este es un prompt más flexible, en lugar del primer prompt que es más estricto y está atento a los detalles, en este el principal objetivo es detectar que cumpla con la finalidad de transmitir correctamente la idea sin tanta importancia en los detalles.</p>

In [59]: PROMPT_JUEZ_FLEXIBLE = """
Eres un educador de primaria evaluando secuencias de pictogramas para niños con
Tu prioridad es determinar si la INTENCIÓN comunicativa global se mantiene, ignora
los detalles.

Asigna un puntaje del 1 al 5:
- 5 o 4: Si un usuario humano promedio podría entender perfectamente la acción p
- 3: Si se entiende la idea central, pero requiere un esfuerzo cognitivo extra o
- 1 o 2: Si el significado se distorsiona tanto que confunde por completo la acc

OBLIGATORIO: Debes responder ÚNICAMENTE con un objeto JSON válido que contenga e
{
 "score": [número entero del 1 al 5],
 "semantic_errors": [lista de strings con errores detectados],
 "missing_concepts": [lista de strings con conceptos del texto original faltant
 "comments": [string con tu justificación detallada de la nota]
}
No agregues texto introductorio ni explicaciones fuera del JSON.
"""

<p>Celda 3: Iteración automática sobre todo tu Dataset (df)</p> <p>Esta celda recorre uno a uno todos los registros de tu DataFrame (los 100-300 ejemplos), extrae los veredictos y los inserta como columnas individuales Nota: Tiene un time.sleep(1.5) para que al usar la API Key gratuita no te salte el bloqueo por límite de peticiones por minuto (RPM).</p>

```
In [60]: # Construir la selección directamente desde los JSON cargados
# Así evitamos depender de un DataFrame intermedio para esta fase

MODEL_TO_EVALUATE = "Modelo 1"
MODEL_FILE = {
    "Modelo 1": "modelo1.json",
    "Modelo 2": "modelo2.json",
    "Modelo 3": "modelo3.json",
    "Modelo 4": "modelo4.json",
}[MODEL_TO_EVALUATE]

test_data_local = load_json("test (2).json")
model_data_local = load_json(MODEL_FILE)

test_data_by_id = {x["id"]: x for x in test_data_local}
model_data_by_id = {x["id"]: x for x in model_data_local}

print(f"JSON de test cargado: {len(test_data_local)} registros")
print(f"JSON del modelo cargado: {len(model_data_local)} predicciones")
print(f"Modelo a evaluar: {MODEL_TO_EVALUATE}")
```

JSON de test cargado: 973 registros
 JSON del modelo cargado: 50 predicciones
 Modelo a evaluar: Modelo 1

```
In [61]: # Listas vacías para recolectar las respuestas de la IA fila por fila
llm_scores = []
llm_errors = []
llm_missing = []
llm_comments = []

import os
import json
from datetime import datetime

# Configuración del modelo y archivos
MODEL_TO_EVALUATE = "Modelo 1"
MODEL_FILE = {
    "Modelo 1": "modelo1.json",
    "Modelo 2": "modelo2.json",
    "Modelo 3": "modelo3.json",
    "Modelo 4": "modelo4.json",
}[MODEL_TO_EVALUATE]
DATA_DIR_LOCAL = globals().get("DATA_DIR", "data")

# Carga local autosuficiente si esta celda se ejecuta en un kernel limpio
if "load_json" not in globals():
    def load_json(filename):
        path = os.path.join(DATA_DIR_LOCAL, filename)
        with open(path, "r", encoding="utf-8") as f:
            return json.load(f)

if "test_data_by_id" not in globals():
    test_data_local = load_json("test (2).json")
    test_data_by_id = {x["id"]: x for x in test_data_local}
else:
    test_data_local = [test_data_by_id[k] for k in sorted(test_data_by_id)]

if "model_data_by_id" not in globals():
    model_data_local = load_json(MODEL_FILE)
```



```

    model_data_by_id = {x["id"]: x for x in model_data_local}
else:
    model_data_local = [model_data_by_id[k] for k in sorted(model_data_by_id)]

available_ids = sorted(model_data_by_id.keys())
print(f"Hay {len(available_ids)} registros disponibles para evaluar.")

# UI interactiva con widgets si están disponibles
try:
    import pandas as pd
    import ipywidgets as widgets
    from IPython.display import display, clear_output

    eval_limit_widget = widgets.BoundedIntText(
        value=5,
        min=1,
        max=len(available_ids),
        description="Registros:",
        style={"description_width": "90px"},
    )

    mode_widget = widgets.ToggleButtons(
        options=[("Aleatorio", "random"), ("IDs", "ids")],
        value="random",
        description="Modo:",
        style={"description_width": "60px"},
    )

    ids_widget = widgets.Textarea(
        value="",
        placeholder="Ej: 13, 17, 25",
        description="IDs:",
        layout=widgets.Layout(width="420px", height="90px"),
        style={"description_width": "60px"},
        disabled=False,
    )

    seed_widget = widgets.IntText(
        value=42,
        description="Semilla:",
        style={"description_width": "70px"},
    )

    run_button = widgets.Button(description="Ejecutar LLM-Judge", button_style="")
    output_area = widgets.Output()

    def run_llm_judge(_):
        with output_area:
            clear_output(wait=True)
            llm_scores.clear()
            llm_errors.clear()
            llm_missing.clear()
            llm_comments.clear()

            eval_limit = max(1, min(int(eval_limit_widget.value), len(available_ids)))
            selection_mode = mode_widget.value

            if selection_mode == "ids":
                selected_ids = []
                raw_ids = ids_widget.value.strip()

```

```

    if raw_ids:
        for item in raw_ids.split(","):
            item = item.strip()
            if not item:
                continue
            try:
                candidate_id = int(item)
            except ValueError:
                print(f"ID inválido ignorado: {item}")
                continue
            if candidate_id in model_data_by_id and candidate_id in
                selected_ids.append(candidate_id)
            else:
                print(f"ID no encontrado en test/modelo: {candidate_
if not selected_ids:
    print("No se ingresaron IDs válidos.")
    return
selected_ids = selected_ids[:eval_limit]
random_state = None
else:
    random_state = int(seed_widget.value)
    selected_ids = list(pd.Series(available_ids).sample(n=eval_limit

print(f"Iniciando evaluación con LLM-Judge sobre {len(selected_ids)}")
print("Se envían a la API: el texto original, la secuencia de pictog
print("No se manda solo el texto ni solo la predicción; se manda el

for position, sample_id in enumerate(selected_ids, start=1):
    print(f"Evaluando ID {sample_id} ({position}/{len(selected_ids)})")
    texto = test_data_by_id[sample_id]["oracion"]
    prediccion = model_data_by_id[sample_id]["output"]

    resultado = evaluar_con_llm_judge(texto, prediccion)
    errores = resultado.get("semantic_errors", [])
    mensaje_error = errores[0] if errores else ""

    llm_scores.append(resultado["score"])
    llm_errors.append(resultado["semantic_errors"])
    llm_missing.append(resultado["missing_concepts"])
    llm_comments.append(resultado["comments"])

    print(f"Progreso: {position}/{len(selected_ids)} | ID {sample_id

    if mensaje_error.startswith("Error de ejecución: 429"):
        print("Cuota agotada. Se detiene la ejecución en esta fila.")
        break

    time.sleep(1.5)

selected_ids_used = selected_ids[:len(llm_scores)]
df_eval = pd.DataFrame({
    "id": selected_ids_used,
    "Text": [test_data_by_id[sid]["oracion"] for sid in selected_ids
    "Prediction": [model_data_by_id[sid]["output"] for sid in select
})

df_eval["score"] = llm_scores
df_eval["semantic_errors"] = llm_errors
df_eval["missing_concepts"] = llm_missing
df_eval["comments"] = llm_comments

```

```

results_dir = os.path.join(DATA_DIR_LOCAL, "llm_judge_results")
os.makedirs(results_dir, exist_ok=True)
stamp = datetime.now().strftime("%Y%m%d_%H%M%S")
output_json_path = os.path.join(
    results_dir,
    f"llm_judge_{MODEL_TO_EVALUATE.replace(' ', '_').lower()}_{stamp}"
)

results_payload = {
    "model": MODEL_TO_EVALUATE,
    "selection_mode": selection_mode,
    "eval_limit": eval_limit,
    "random_state": random_state,
    "selected_ids": selected_ids_used,
    "results": df_eval.to_dict(orient="records"),
}

with open(output_json_path, "w", encoding="utf-8") as f:
    json.dump(results_payload, f, ensure_ascii=False, indent=2)

print("\n¡Evaluación completada!")
print(f"Archivo JSON guardado en: {output_json_path}")
display(df_eval)

run_button.on_click(run_llm_judge)

display(widgets.VBox([
    widgets.HTML("<b>Configura la evaluación antes de ejecutar:</b>"),
    eval_limit_widget,
    mode_widget,
    ids_widget,
    seed_widget,
    run_button,
    output_area,
]))

except Exception:
    # Fallback a input() si widgets no están disponibles
    print("ipywidgets no está disponible; se usará modo texto.")
    print("Regresa a la versión anterior de la celda si prefieres prompts de con

```

Hay 50 registros disponibles para evaluar.

VBox(children=(HTML(value='Configura la evaluación antes de ejecutar:'), BoundedIntText(value=5, descri...

<p>Celda 4: Experimento Opcional de Consistencia (Variando Prompts) Realizar experimentos variando prompts y comparando consistencia. En esta celda se puede evaluar rápidamente un mismo ejemplo usando un enfoque pedagógico/flexible y contrastarlo con el enfoque analítico anterior.</p>

<h2 id="Fase-6:-An%C3%A1lisis-de-errores">Fase 6: Análisis de errores</h2> <h3

id="BLEU-alto-pero-mala-calidad-humana">BLEU alto pero mala calidad humana</h3>

<h3 id="BLEU-bajo-pero-buena-calidad-humana">BLEU bajo pero buena calidad

humana</h3> <p>Identificamos ejemplos donde BLEU y la evaluación humana divergen, lo que revela las limitaciones de BLEU como métrica de calidad semántica en sistemas de generación de pictogramas.</p>

```

In [65]: # — Fase 6: Análisis de errores —————
#
# Caso 1: BLEU alto (>50) + score humano bajo (<3)
#         → BLEU no detecta errores semánticos que sí detecta el humano
# Caso 2: BLEU bajo (<20) + score humano alto (>=4)
#         → BLEU penaliza variaciones válidas que el humano acepta
# —————

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

# — Construir DataFrame unificado métricas + humano —————
error_rows = []

for m_name in ["Modelo 1", "Modelo 2", "Modelo 3", "Modelo 4"]:
    df_m = per_sample[m_name].copy()
    h_mean = human_mean_sample[m_name]

    for sid in df_m.index:
        if sid not in h_mean.index:
            continue
        error_rows.append({
            "id": sid,
            "Modelo": m_name,
            "BLEU": df_m.loc[sid, "BLEU"],
            "chrF++": df_m.loc[sid, "chrF++"],
            "Concept_F1": df_m.loc[sid, "Concept_F1"],
            "Coverage": df_m.loc[sid, "Coverage"],
            "Sem_Sim": df_m.loc[sid, "Semantic Similarity"],
            "score_humano": h_mean.loc[sid],
            "oracion": test_by_id[sid]["oracion"],
        })

df_error = pd.DataFrame(error_rows)

# — Umbrales —————
BLEU_ALTO = 50
BLEU_BAJO = 20
HUM_BAJO = 3
HUM_ALTO = 4

# — Caso 1: BLEU alto + calidad humana baja —————
df_caso1 = df_error[
    (df_error["BLEU"] > BLEU_ALTO) &
    (df_error["score_humano"] < HUM_BAJO)
].sort_values("BLEU", ascending=False)

print(f"== Caso 1: BLEU alto (>{BLEU_ALTO}) + score humano bajo (<{HUM_BAJO}) =")
print(f"Total casos: {len(df_caso1)}")
cols = ["Modelo", "BLEU", "chrF++", "Concept_F1", "Coverage", "Sem_Sim", "score_humano"]
print(df_caso1[cols].to_string(index=True))

# — Caso 2: BLEU bajo + calidad humana alta —————
df_caso2 = df_error[
    (df_error["BLEU"] < BLEU_BAJO) &
    (df_error["score_humano"] >= HUM_ALTO)
].sort_values("BLEU", ascending=True)

```

```

print(f"\n== Caso 2: BLEU bajo (<{BLEU_BAJO}) + score humano alto (>={HUM_ALTO})")
print(f"Total casos: {len(df_caso2)}")
print(df_caso2[cols].to_string(index=True))

# — Análisis cualitativo top 3 de cada caso —————
for label, df_caso in [
    ("Caso 1 – BLEU alto / calidad baja", df_caso1),
    ("Caso 2 – BLEU bajo / calidad alta", df_caso2),
]:
    print(f"\n== Análisis cualitativo: {label} ==")
    for _, row in df_caso.head(3).iterrows():
        sid = int(row["id"])
        hyp = models[row["Modelo"]][sid]["output"]
        ref = test_by_id[sid]["traduccion"]
        print(f"\n ID {sid} | {row['Modelo']}")
        print(f" Oración : {row['oracion']}")
        print(f" Referencia : {ref}")
        print(f" Predicción : {hyp}")
        print(f" BLEU={row['BLEU']:.1f} chrF++={row['chrF++']:.1f} "
              f"ConceptF1={row['Concept_F1']:.1f} Humano={row['score_humano']:.1f}")
        print(" " + "-"*55)

# — Visualización —————
fig, axes = plt.subplots(1, 2, figsize=(14, 5))
colors_mod = sns.color_palette("husl", 4)
modelo_color = {m: c for m, c in zip(
    ["Modelo 1", "Modelo 2", "Modelo 3", "Modelo 4"], colors_mod)}

for ax, df_caso, title, color in [
    (axes[0], df_caso1, f"Caso 1: BLEU alto (>{BLEU_ALTO}) +\nscore humano bajo",
    (axes[1], df_caso2, f"Caso 2: BLEU bajo (<{BLEU_BAJO}) +\nscore humano alto
]:
    if df_caso.empty:
        ax.text(0.5, 0.5, "Sin casos con estos umbrales",
            ha="center", va="center", transform=ax.transAxes,
            fontsize=11, color="gray")
        ax.set_title(title, fontweight="bold")
        continue

    for mod in ["Modelo 1", "Modelo 2", "Modelo 3", "Modelo 4"]:
        sub = df_caso[df_caso["Modelo"] == mod]
        if not sub.empty:
            ax.scatter(sub["BLEU"], sub["score_humano"],
                label=mod, color=modelo_color[mod],
                s=80, alpha=0.8, edgecolors="white")

    ax.axvline(BLEU_ALTO if "alto" in title else BLEU_BAJO,
        color=color, linestyle="--", alpha=0.6)
    ax.axhline(HUM_BAJO if "bajo" in title else HUM_ALTO,
        color=color, linestyle="--", alpha=0.6)
    ax.set_xlabel("BLEU per-sample", fontsize=11)
    ax.set_ylabel("Score humano promedio", fontsize=11)
    ax.set_title(title, fontweight="bold")
    ax.legend(fontsize=8)
    ax.grid(alpha=0.3)

plt.suptitle("Fase 6: Análisis de errores – divergencia BLEU vs evaluación human
    fontsize=13, fontweight="bold")
plt.tight_layout()

```

```
plt.savefig(" analisis_errores_bleu_humano.png", dpi=150, bbox_inches="tight")  
plt.show()  
print("Guardado: analisis_errores_bleu_humano.png")
```

== Caso 1: BLEU alto (>50) + score humano bajo (<3) ==

Total casos: 24

	Modelo	BLEU	chrF++	Concept_F1	Coverage	Sem_Sim	score_h
umano					oracion		
15	Modelo 1	100.000000	100.000000	100.000000	100.000000	26.216053	2.5
71429					inflamación		
23	Modelo 1	100.000000	100.000000	100.000000	100.000000	93.750931	2.8
57143					energía química		
115	Modelo 3	100.000000	100.000000	100.000000	100.000000	26.216053	2.8
57143					inflamación		
123	Modelo 3	100.000000	100.000000	100.000000	100.000000	93.750931	2.8
57143					energía química		
165	Modelo 4	100.000000	100.000000	100.000000	100.000000	26.216053	2.5
71429					inflamación		
173	Modelo 4	100.000000	100.000000	100.000000	100.000000	93.750931	2.8
57143					energía química		
170	Modelo 4	75.983569	63.492489	50.000000	50.000000	85.288651	1.2
85714					Té de hierbas		
80	Modelo 2	75.983569	60.869129	50.000000	50.000000	65.990585	2.0
00000					Mis fortalezas		
130	Modelo 3	75.983569	64.566133	50.000000	50.000000	92.572357	2.0
00000					Mis fortalezas		
177	Modelo 4	67.865027	87.313064	85.714286	100.000000	87.079124	1.8
57143					Si la persona está viva siente hambre		
116	Modelo 3	66.063286	69.701744	66.666667	66.666667	83.906647	2.4
28571					Escribimos una noticia		
67	Modelo 2	61.153806	57.598318	44.444444	50.000000	37.428635	1.2
85714					La paleontóloga descubre los huesos de dinosaurio		
8	Modelo 1	60.155766	61.834521	60.000000	60.000000	61.912392	1.8
57143					Esa fábrica producía cosas sin parar		
58	Modelo 2	60.155766	61.834521	60.000000	60.000000	55.970890	2.7
14286					Esa fábrica producía cosas sin parar		
194	Modelo 4	58.739491	89.485845	80.000000	100.000000	89.653442	2.7
14286					Explicar el orden		
90	Modelo 2	57.995688	66.641203	57.142857	66.666667	76.391563	1.7
14286					Es una piedra corazón		
54	Modelo 2	55.032121	35.395353	0.000000	0.000000	63.751137	1.2
85714					Tallo		
81	Modelo 2	55.032121	43.044218	0.000000	0.000000	43.354317	1.2
85714					Huevos		
108	Modelo 3	53.036246	51.378006	40.000000	40.000000	72.370407	1.1
42857					Esa fábrica producía cosas sin parar		
152	Modelo 4	53.036246	56.335586	40.000000	40.000000	82.078300	2.8
57143					Yo quiero usar la polea para mover los objetos		
176	Modelo 4	53.036246	51.771226	44.444444	40.000000	78.052734	2.7
14286					Igual que a mí cuando era un bebé		
193	Modelo 4	52.212571	51.232877	44.444444	40.000000	65.780403	1.8
57143					Enviamos un mensaje fácil y corto		
24	Modelo 1	52.025569	61.128016	44.444444	50.000000	89.261177	2.8
57143					El corazón mueve la sangre por el cuerpo		
76	Modelo 2	51.742604	68.178616	50.000000	60.000000	90.816628	1.7
14286					Igual que a mí cuando era un bebé		

== Caso 2: BLEU bajo (<20) + score humano alto (>=4) ==

Total casos: 1

	Modelo	BLEU	chrF++	Concept_F1	Coverage	Sem_Sim	score_humano
oracion							
37	Modelo 1	16.233396	22.478398	0.0	0.0	86.340508	4.428571
	Subimos al autobús						

== Análisis cualitativo: Caso 1 – BLEU alto / calidad baja ==

ID 15 | Modelo 1

Oración : inflamación

Referencia : pict_31833

Predicción : pict_31833

BLEU=100.0 chrF++=100.0 ConceptF1=100.0 Humano=2.57

ID 23 | Modelo 1

Oración : energía química

Referencia : pict_26059 pict_36374

Predicción : pict_26059 pict_36374

BLEU=100.0 chrF++=100.0 ConceptF1=100.0 Humano=2.86

ID 15 | Modelo 3

Oración : inflamación

Referencia : pict_31833

Predicción : pict_31833

BLEU=100.0 chrF++=100.0 ConceptF1=100.0 Humano=2.86

== Análisis cualitativo: Caso 2 – BLEU bajo / calidad alta ==

ID 960 | Modelo 1

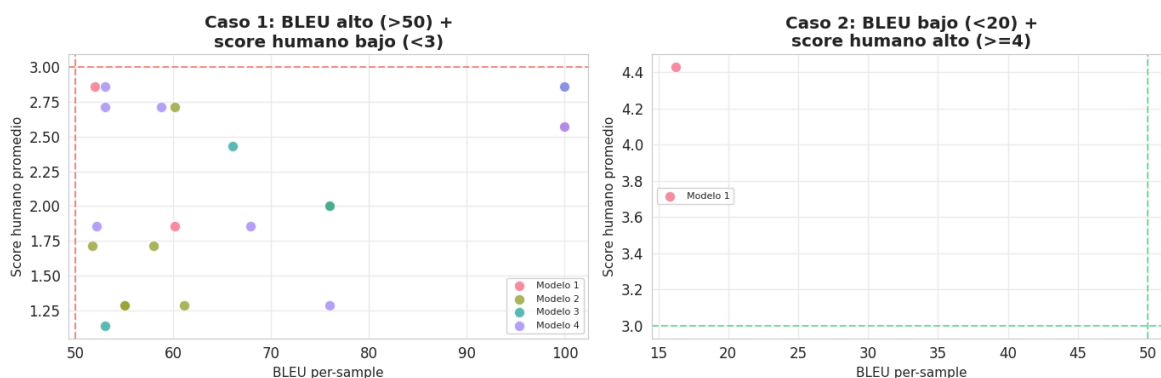
Oración : Subimos al autobús

Referencia : pict_36594

Predicción : pict_6617 pict_2262

BLEU=16.2 chrF++=22.5 ConceptF1=0.0 Humano=4.43

Fase 6: Análisis de errores — divergencia BLEU vs evaluación humana



Guardado: analisis_errores_bleu_humano.png

Conclusiones: Análisis de errores

>BLEU alto + calidad humana baja (24 casos)</h3> BLEU puede ser perfecto (100) mientras el evaluador humano asigna scores bajos (~2.5). Esto ocurre principalmente en oraciones de **una sola palabra** como "inflamación" o "energía química" — el modelo acierta el único token pero el evaluador considera el concepto difícil de comunicar con pictogramas. También aparecen casos donde BLEU ~60–75 con ConceptF1 alto pero el humano penaliza el orden o el contexto semántico que BLEU no captura.</p> >BLEU bajo + calidad humana alta (1 caso)</h3> Solo se encontró 1 caso (Modelo 1, BLEU=16.2, humano=4.43). El modelo generó tokens semánticamente válidos pero distintos a la referencia exacta, que

BLEU penalizó fuertemente. Esto sugiere que en este dataset los casos donde el humano acepta variaciones que BLEU rechaza son poco frecuentes.

Implicación general

BLEU no es una métrica confiable para evaluar calidad semántica en sistemas AAC de manera aislada. Los 24 casos de B3.1 confirman que es necesario complementarlo con Concept F1, Similitud Semántica y evaluación humana para obtener una visión completa de la calidad del sistema.

9. Resumen y próximos pasos

```
In [63]: # — Tabla resumen ejecutiva —————
print("=" * 65)
print(" RESUMEN SEMANA 4 - EVALUACIÓN SEMÁNTICA DE PICTOGRAMAS")
print("=" * 65)

print("\nDataset:")
print(f" Train: {len(train_data):,} | Test: {len(test_data):,} | Val: {len(val_d)}")
print(f" 50 ejemplos evaluados con 4 modelos y 8 anotadores humanos\n")

print("Métricas automáticas:")
for _, row in df_all_metrics.iterrows():
    print(
        f" {row['Modelo']}: "
        f"BLEU={row['BLEU']:.1f} "
        f"chrF++={row['chrF++']:.1f} "
        f"ConceptF1={row['Concept F1']:.1f} "
        f"Coverage={row['Coverage']:.1f}"
    )

print("\nConcordancia inter-anotador (Krippendorff α):")
for _, row in df_alpha.iterrows():
    a = row["Krippendorff α"]
    interp = "Fuerte" if a >= 0.8 else ("Moderado" if a >= 0.6 else "Débil")
    print(f" {row['Modelo']}: α={a} ({interp})")

print("\nCorrelación más alta (Pearson):")
top_corr = df_corr.loc[df_corr["Pearson r"].abs().idxmax()]
print(f" {top_corr['Modelo']} - {top_corr['Métrica']}: r={top_corr['Pearson r']}:")

print("\nPróximos pasos (Semana 5-7):")
print(" 1. LLM-Judge: diseñar prompts de evaluación estructurada")
print(" 2. Similitud semántica con embeddings")
print(" 3. Plataforma web de evaluación humana (FastAPI + React)")
print(" 4. Análisis de errores: BLEU alto vs. baja calidad")
print(" 5. Reporte técnico en formato paper")
```

RESUMEN SEMANA 4 – EVALUACIÓN SEMÁNTICA DE PICTOGRAMAS

Dataset:
Train: 15,208 | Test: 973 | Val: 973
50 ejemplos evaluados con 4 modelos y 8 anotadores humanos

Métricas automáticas:
Modelo 1: BLEU=41.2 chrF++=55.7 ConceptF1=44.5 Coverage=48.8
Modelo 2: BLEU=29.7 chrF++=43.4 ConceptF1=25.5 Coverage=27.2
Modelo 3: BLEU=32.3 chrF++=48.8 ConceptF1=36.6 Coverage=40.1
Modelo 4: BLEU=30.1 chrF++=41.9 ConceptF1=28.9 Coverage=30.6

Concordancia inter-anotador (Krippendorff α):
Modelo 1: α =0.1136 (Débil)
Modelo 2: α =0.3598 (Débil)
Modelo 3: α =0.2208 (Débil)
Modelo 4: α =0.1661 (Débil)

Correlación más alta (Pearson):
Modelo 2 – Semantic Similarity: r =0.5005

- Próximos pasos (Semana 5-7):
- 1. LLM-Judge: diseñar prompts de evaluación estructurada
 - 2. Similitud semántica con embeddings
 - 3. Plataforma web de evaluación humana (FastAPI + React)
 - 4. Análisis de errores: BLEU alto vs. baja calidad
 - 5. Reporte técnico en formato paper

<h2 id="%F0%9F%93%81-Archivos-generados"> 📁 Archivos generados</h2> <thead>
<th> 📄 Archivo</th> <th> 📄 Descripción</th> </thead> <tbody> </tbody>
<tbody></tbody>

dist_longitudes.png	Histogramas de longitud de frases y pictogramas
top_pictogramas.png	Top 20 pictogramas más frecuentes
metricas_baseline.png	BLEU y chrF++ por modelo
heatmap_metricas.png	Todas las métricas en heatmap
boxplot_scores_humanos.png	Distribución de scores humanos
correlacion_metricas.png	Heatmaps de correlación Pearson/Spearman
scatter_correlacion.png	Scatter plots métrica vs humano